A Lightweight Trusted Execution Environment for Low-end Programmable Logic Controllers

Abstract

Low-end Programmable Logic Controllers (LE-PLCs) are extensively utilized in the control loops of Industrial Control Systems (ICSs) for their simplicity and operational efficiency. Due to their high reliability requirements and limited computational resources, LE-PLCs face significant challenges in integrating robust security measures, rendering them vulnerable to serious threats. Trusted Execution Environment (TEE) is a cutting-edge security isolation mechanism offering robust protective capabilities; however, existing TEE solutions often require substantial hardware modifications or extensive software instrumentation, making them impractical for LE-PLCs.

To overcome these limitations, we propose LiTEE-PLC, a lightweight Trusted Execution Environment specifically designed for LE-PLCs. LiTEE-PLC partitions LE-PLC software modules into secure and non-secure worlds based on defined isolation criteria and an established threat model. By leveraging the built-in Memory Protection Unit (MPU) hardware component in LE-PLCs to set different memory access permissions, it realizes efficient isolation between the two worlds with minimal software instrumentation and no hardware modifications. Moreover, LiTEE-PLC employs a selective protection mechanism based on the Minimum Trusted I/O Path, together with an identity-authentication triggering mechanism based on state estimation, thereby achieving peripheral access control and network access control with low resource overhead. Our evaluation demonstrates that LiTEE-PLC imposes minimal overhead on system resources. Specifically, LiTEE-PLC achieves an average runtime overhead of 0.98%, with SRAM usage increasing by 2.31% and FLASH usage rising by 1.82%.

CCS Concepts

• **Security and privacy → Embedded systems security**.

Keywords

Trusted Execution Environment, Programmable Logic Controller, Memory Protection Unit, Industrial Control Systems

ACM Reference Format:

. 2018. A Lightweight Trusted Execution Environment for Low-end Programmable Logic Controllers. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 18 pages. https://doi.org/XXXXXXX.XXXXXXX

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

~~Low-end programmable logic controllers [43](LE-PLCs [27], a.k.a. Low-cost PLCs or single-process PLCs [28]), are class of economical, functionally-specific programmable logic controller (PLC) which is widely deployed in industrial control systems (ICS) [25, 39, 43, 44, 48]. These devices are widely adopted in simple control loops, because of their efficiency and affordability [2, 13, 33]. Similar to traditional PLCs, the operational model of LE-PLCs generally adheres to a cyclic scanning mechanism, which sequentially performs input acquisition, program execution, output updating, and system communication to ensure both reliability and real-time performance of the control loops.~~[old]

Bare-metal programmable logic controllers (PLCs), including low-cost and single-process PLCs [27, 28, 43], are widely deployed in industrial control systems (ICSs) [25, 39, 43, 44, 48]. These devices are adopted in simple and cost-sensitive control loops because of their efficiency and affordability [2, 13, 33]. Similar to traditional PLCs, they execute a cyclic scan that sequentially performs input acquisition, program execution, output updating, and system communication to maintain the reliability and real-time performance of control loops. With the advancement of information technologies, ICS architectures have gradually evolved from isolated systems into interconnected industrial networks. External operator access, engineering-station programming, remote maintenance, and file or program transfer have therefore become routine interactions with PLCs, making traditional internal network isolation insufficient as the only defense.[rev]

~~With the advancement of information technologies, ICS architectures have gradually evolved from isolated systems into an interconnected Industrial Internet. Consequently, traditional internal network isolation is no longer sufficient to defend against expanded attack surface such as legitimate external access, remote file transfers, remote maintenance. This makes LE-PLCs vulnerable to various of emerging threats, such as parameter tampering attacks [20], control program injection [53], and false data injection [22]. LE-PLCs were designed to be primarily focused on reliability and real-time performance, often omitting the internal deployment of security mechanisms. However, being the core component in control loops, any compromise of an LE-PLC directly impairs the stability of physical systems, leading to reduced productivity, system shutdowns, or even casualties. Therefore, implementing effective security measurements without disrupting regular LE-PLC operations is critical for ensuring the security of ICS.~~[old]

This connectivity turns communication and maintenance functions into attack paths to the physical process. Parameter tampering may change set points or runtime variables and drive the controller beyond safe operating ranges [20]. Control-program injection may replace or modify the decision logic, causing the PLC to issue malicious or unsafe actuator commands [53]. False data injection may forge sensor values and mislead the controller into making incorrect feedback decisions [22]. Because PLCs directly mediate sensing and actuation, these attacks can propagate from software compromise

to physical consequences, including quality degradation, equipment damage, process shutdown, environmental incidents, and safety hazards. Therefore, effective security isolation must protect the control path without disrupting scan-cycle timing or the deterministic behavior expected in industrial deployments.[rev]

~~The Trusted Execution Environment (TEE) has emerged as a promising security solution in recent years [35, 42, 46]. It divides the computing system into a secure world and a non-secure world, enforcing strict access control through memory isolation. TEE implementations fall into two categories: hardware-based and software-based. The former, such as ARM TrustZone [6], Intel SGX [19], and SANCUS [45], run sensitive code in physically isolated memory with specialized hardware protection. The latter, such as PISTIS [24] and SV [3], instrument non-secure code to monitor memory access and trigger verification. These technologies can effectively mitigate software-level attacks from the non-secure world [35], establishing a trusted runtime environment for critical applications. For LE-PLC, which directly manipulates physical systems, integrating it with TEE offers a promising path to enhance the ICS security.~~[old]

Memory-based security isolation, including trusted-execution-environment designs, provides a promising direction for protecting critical code and data [35, 42, 46]. Existing designs, however, are not a direct fit for bare-metal PLCs. Hardware isolation mechanisms such as ARM TrustZone [6], Intel SGX [19], and SANCUS [45] can provide strong isolation, but they assume specific hardware support or platform architectures that may not be available on deployed microcontroller-class PLCs. Software-only isolation mechanisms such as PISTIS [24] and SV [3] preserve compatibility with legacy hardware, but their instruction-level checks or virtualization logic can increase verification frequency and code complexity. Thus, the problem is not whether hardware-assisted or software-assisted isolation can work in general, but how to obtain sufficient isolation for bare-metal PLCs by using hardware already present on such platforms and by exploiting the deterministic structure of PLC execution.[rev]

~~Nevertheless, LE-PLCs impose higher requirements on real-time performance and reliability, rendering existing TEE solutions less suitable for direct implementation. Hardware TEEs require modifications to the underlying architecture, which adds hardware complexity and may undermine reliability, imposing higher costs and uncertainties on ICS enterprises. Software TEEs, on the other hand, instrument security-check code into the non-secure world code to monitor each memory access, which also entails considerable resource overhead, making it difficult to meet the real-time requirements of LE-PLCs.~~[old]

~~Despite these challenges, LE-PLCs exhibit the following characteristics that enable the development of lightweight TEE solutions. First, LE-PLCs predominantly utilize microcontroller architectures rather than x86 or high-performance ARM architectures which widely incorporate a hardware component called the Memory Protection Unit (MPU) [23]. As a memory protection mechanism designed to prevent software faults, the MPU has potential to facilitate memory isolation and access control for TEEs. Second, LE-PLCs generally follow minimalistic design principles [48] and often operate in a bare-metal environment (i.e., without an operating system), where all software modules operate in privileged mode and share the entire memory space [17]. This architecture enables bottom-up physical memory region partitioning and security isolation construction. Third, LE-PLCs strictly adhere to cyclic scanning operational patterns and have relatively singular functional responsibilities, making their MMIO access behaviors and I/O data changes highly predictable, enabling the reduction of verification frequency for normal access behaviors based on established behavioral baselines.~~[old]

The key problem addressed in this paper is memory and peripheral isolation for bare-metal PLCs whose software modules originally run in privileged mode and share a unified address space. This setting is different from OS-based systems, where processes, drivers, and user tasks can be isolated through mature kernel abstractions. In a bare-metal PLC, the runtime, protocol stack, control program, I/O buffers, and MMIO drivers are commonly linked into one firmware image. A compromise of externally exposed code can therefore lead to unauthorized reads or writes of control logic, I/O images, set points, or safety-critical peripherals. At the same time, PLCs provide an opportunity for lightweight isolation: their cyclic scan order is stable, their I/O data paths are explicit, and their MMIO interactions are usually fixed by the deployed control scenario. These properties make it possible to identify the trusted control path and reduce unnecessary runtime checks.[rev]

~~This paper proposes a Lightweight Trusted Execution Environment for Low-end Programmable Logic Controllers (LiTEE-PLC), which is a hardware-assisted, comprehensive, and lightweight TEE solution that effectively defends against software-based security threats faced by LE-PLCs while meeting their reliability and real-time requirements. To the best of our knowledge, this is the first work that systematically explores how to leverage the MPU hardware component to enhance TEE capabilities. The major works include: 1) Establishing the threat model and isolation criteria for LE-PLCs, followed by a fine-grained division of LE-PLC software modules into secure and non-secure worlds, thereby logically segregating core components from potential attack surface; 2) Leveraging the built-in MPU hardware component of LE-PLCs to manage access permissions for different memory regions, enabling memory isolation between the two worlds with minimal software modifications and no hardware modifications; 3) Propose a selective protection mechanism based on the Minimum Trusted I/O Path. By combining fine-grained program slicing with periodic heartbeat-based execution attestation, it precisely verifies accesses to shared peripherals and critical trusted data, thereby enabling peripheral access control with reduced verification frequency; 4) Proposing lightweight network access control by calculating the deviation between LE-PLC input data from network sensor and the expected physical system state, triggering identity authentication for field devices whenever input data persistently deviates from expectations, thereby mitigating false data injection attacks with reduced authentication overhead.~~[old]

This paper presents LiTEE-PLC, an MPU-assisted security isolation framework for bare-metal PLCs. LiTEE-PLC does not aim to build a general-purpose TEE. Instead, it uses PLC-specific execution semantics to construct two protection domains: a trusted domain that contains the control program, I/O images, trusted runtime services, and safety-critical MMIO paths; and an untrusted domain that contains externally exposed communication modules and protocol processing. The framework then configures the MPU to enforce memory and peripheral permissions between the domains and uses controlled software entry points only where cross-domain interaction is necessary. For shared I/O operations, LiTEE-PLC further protects the minimum trusted I/O

path and uses I/O-data-driven triggers to invoke stronger verification only when the observed process behavior becomes suspicious.[rev]

The contributions of this paper are as follows:

(1) **A hardware-assisted TEE architecture targeted at LE-PLCs.** To the best of our knowledge, this paper presents the first attempt to construct a hardware-assisted TEE architecture for LE-PLCs by leveraging the inherent MPU hardware component. We perform fine-grained decoupling of software modules into two distinct worlds and systematically investigate the utilization of MPU for enabling robust memory isolation and access control between the two worlds.[old]

(2) **MPU-based memory isolation.** MPU is utilized to preset access permissions for the non-secure world across various memory regions, thereby effectively blocking unauthorized access behaviors at the hardware level and eliminating the need for instrumentation verification for most of the access behaviors.[old]

(3) **Lightweight access control mechanisms for LE-PLCs.** Lightweight peripheral and network access controls are proposed based on MPU fault triggering and state estimation, ensuring comprehensive control of access behaviors across the two worlds while minimizing the frequency of both peripheral verification and identity authentication.[old]

(4) **Experimental validation and performance evaluation.** Based on multi-scenario benchmark testing, results show that LiTEE-PLC demonstrates outstanding lightweight advantages while protecting against all major threats of PLCs, achieving an average runtime overhead of 0.98%, a 2.31% increase in SRAM usage, and a 1.82% increase in FLASH usage.[old]

(1) **An I/O-aware partitioning method for bare-metal PLCs.** We propose a method for dividing PLC firmware into trusted and untrusted domains according to cyclic scan semantics, fixed MMIO usage, and I/O data dependencies. Rather than relying on a coarse division by software modules alone, the method identifies the minimum trusted I/O path that affects physical actuation and places the control program, I/O images, safety-critical runtime logic, and trusted peripheral operations in the trusted domain. Externally exposed communication and protocol-processing components are kept in the untrusted domain to reduce the attack surface of the trusted control path.[rev]

(2) **An MPU-based management method for the two domains.** We show how to use the built-in MPU to enforce the above partition on bare-metal PLCs without hardware modification. LiTEE-PLC maps trusted code, trusted data, untrusted code, untrusted data, and peripheral regions into MPU-controlled address ranges, runs the untrusted domain in non-privileged mode, and relies on default-deny background-region behavior to block unauthorized access to trusted memory and safety-critical peripherals. Necessary cross-domain calls are routed through controlled software entry points, so most ordinary memory accesses no longer require full software instrumentation.[rev]

(3) **An I/O-data-driven cross-domain access-control method.** We design lightweight runtime checks for the I/O interactions that cannot be completely separated. For trusted outputs sent through shared peripherals, LiTEE-PLC protects the minimum trusted I/O path and uses heartbeat-based execution attestation to detect skipped or forged sending operations. For networked sensor inputs, LiTEE-PLC estimates the expected process state and triggers stronger data-source verification only when input deviations persist within a sliding window. This design focuses on I/O-data-driven protection rather than claiming to solve general identity authentication or general access control.[rev]

The remainder of this paper is organized as follows: Chapter 2 reviews related work and establishes the context for our research; Chapter 3 elaborates on the theoretical foundations and threat model; Chapter 4 details the architecture design of LiTEE-PLC; Chapter ?? introduces the details of system implementation; Chapter ?? presents a thorough security analysis; Chapter 5 discusses performance evaluation results; Chapter 6 highlights future research directions; and Chapter 7 concludes with a summary of the research contributions.[old]

The remainder of this paper is organized as follows. Section 3 presents the background and threat model. Section 4 describes the design of LiTEE-PLC. Section ?? introduces the implementation details. Section ?? provides the security analysis. Section 5 evaluates performance and resource overhead. Section 6 discusses limitations and future directions. Section 2 reviews related work, and Section 7 concludes the paper.[rev]

The Related Work section was originally placed immediately after the Introduction.[old]

The Related Work section is moved to the end of the paper, before the Conclusion, so that the problem statement, design, implementation, and evaluation are presented before positioning LiTEE-PLC against prior work.[rev]

In this paper, we propose a hardware-assisted, lightweight TEE solution with comprehensive protection capabilities. Based on the predictable behavior and streamlined hardware architecture of LE-PLCs, we systematically investigate the application of the MPU in constructing TEEs. This includes fine-grained decoupling of LE-PLC software modules to clarify resource access in two worlds, configuring MPU memory regions to set access permissions and using local instrumentation to construct the boundary between worlds, as well as leveraging the Minimum Trusted I/O Path to achieve comprehensive peripheral access control while minimizing verification frequency. Compared to hardware TEEs, our solution avoids modifications to the underlying architecture; compared to software-based TEEs, it significantly reduces the need for frequent instrumentation checks. Moreover, mitigate the false data injection attacks originated by networked sensors, we derive inspiration from anomaly based dynamic access control [34] and introduce a lightweight network access control scheme based on state estimation. In particular, we optimize the design of the sliding window in the anomaly detection module, effectively defending against data injection attacks while reducing false alarms that trigger identity authentication events, thereby guarantee triggering authentication against all kinds of data injection behaviors and minimizing false alarms that could trigger unnecessary authentication.[old]

This paper presents LiTEE-PLC, an MPU-assisted security isolation framework for bare-metal PLCs. LiTEE-PLC does not aim to build a general-purpose trusted execution environment. Instead, it uses PLC-specific execution semantics to construct two protection domains: a trusted domain that contains the control program, I/O images, trusted runtime services, and safety-critical MMIO paths; and an untrusted domain that contains externally exposed communication modules and protocol processing. The framework configures the MPU to enforce memory and peripheral permissions between the domains and uses controlled software entry points only where cross-domain interaction is necessary. For shared I/O operations, LiTEE-PLC protects the minimum trusted I/O path and uses I/O-data-driven triggers to invoke stronger verification only when the observed process behavior becomes suspicious.[rev]

Table 1: Comparison between LiTEE-PLC and state-of-the-art protection methods for Low-End Embedded Devices

Name	No Hardware Modification	Memory Isolation Verification Frequency	Target Firmware	Peripheral Access Control Support	Peripheral Access Control Verification Frequency	Network Access Control Support	Network Access Control Verification Frequency	Attack Model
SANCUS [45]	×	Infrequent	RTOS	✓	Frequent	□	Frequent	Weak†
TyTAN[15]	×	Infrequent	RTOS	✓	Frequent	□	Frequent	Weak†
SμV[3]	✓	Frequent	Bare-metal	✓	Frequent	×	×	Weak†
PISTIS[24]	✓	Frequent	Bare-metal	✓	Frequent	×	×	Weak†
M2MON[30]	✓	Infrequent	RTOS	✓	Frequent	×	×	Weak†
μBOX[55]	✓	Moderate	RTOS + Bare-metal	×	×	×	×	Weak†
LiTEE-PLC	✓	**Moderate**	**Bare-metal**	✓	**Infrequent**	✓	**Infrequent**	**Strong**

✓: Support □: Partial Support ×: No Support
†**Weak**: Does not consider special threats in industrial environments, e.g., false data injection attacks and malware present on the device.

2 Related Work

Table 1 compares LiTEE-PLC with representative isolation and protection mechanisms for resource-constrained embedded controllers.[rev]

Hardware-assisted isolation and mitigation. ~~In recent years, various hardware TEEs have been widely adopted to achieve secure isolation. ARM TrustZone [6, 7] achieves secure hardware-level isolation by partitioning the system into a secure world and a non-secure world. For instance, TEE-PLC introduced high-frequency verification for network access control in advanced PLC systems [36]). Intel SGX [19] facilitates application-level isolation, allowing code and data to execute within encrypted enclaves, offering strong resistance against compromise even during operating system breaches. AMD SEV [29] enhances cloud security by encrypting virtual machine memory to protect data confidentiality. While these hardware mechanisms provide a solid foundation for trusted computing, they often rely on complex and costly hardware features. Various academic proposals have also introduced hardware TEEs suitable for resource-constrained devices to mitigate these high costs. Sancus [45] introduces a cost-efficient, fully hardware-based trusted computing base designed to provide remote attestation and inter-module communication. TrustLite [32] employs memory access controls related to execution to isolate embedded tasks and ensure secure interrupt handling. TyTAN [15] extends TrustLite with support for dynamic task loading and secure inter-task communication, emerging as the first TEE architecture to provide dynamic roots of trust alongside real-time guarantees for resource-constrained devices. GAROTA [1] introduces a trust anchor for low-end microcontrollers (MCUs) that guarantees the execution of critical functions upon associated trigger events. However, hardware-based TEEs often require significant architectural redesigns, leading to challenges such as heightened complexity, diminished reliability, high costs from hardware upgrades, and reduced flexibility due to time constraints. As specialized devices in industrial control systems, LE-PLCs impose stringent requirements for real-time performance and reliability. As a result, industrial enterprises are hesitant to replace existing LE-PLC devices and often avoid procuring systems with overly complex hardware designs. Moreover, these solutions fail to optimize verification frequency for peripheral and network access control, requiring verification of numerous legitimate accesses, which exacerbates runtime overhead issues in LE-PLC systems.[old]~~

Hardware-assisted isolation mechanisms protect critical code and data by enforcing access-control decisions below ordinary application logic. ARM TrustZone [6, 7] separates execution into secure and non-secure states. Intel SGX [19] protects application enclaves with enclave metadata, controlled entry/exit, and memory-encryption support. AMD SEV [29] protects virtual machines by encrypting guest memory. A PLC-oriented TrustZone-based design has also been used to mediate network access in advanced PLC platforms [36]. In addition, Sancus [45], TrustLite [32], TyTAN [15], and GAROTA [1] provide hardware roots of trust, protected module execution, secure interrupt handling, remote attestation, or trigger-based execution guarantees for constrained devices. These mechanisms are effective when the target platform already provides the required security extensions. However, their hardware assumptions are not always satisfied by deployed microcontroller-class PLCs. The required features may include separated security states, secure-monitor transitions, enclave metadata, memory-encryption engines, attestation roots, protected inter-module control logic, or dedicated bus/peripheral firewalls. Integrating such features into an existing PLC product line may require a different SoC, a redesigned firmware boot and privilege model, and renewed validation of peripheral interactions. LiTEE-PLC therefore takes a narrower path: it uses the MPU that is already available on many microcontroller-class PLC platforms and combines MPU permissions with PLC-specific I/O semantics, instead of assuming enclave memory encryption, a separate secure monitor, or new peripheral-security hardware.[rev]

~~Many studies have attempted to leverage existing hardware features to mitigate attacks on embedded systems. For example, EPOXY [17] and M2MON [30] identify privileged instructions at compile time and instrument their access to peripherals. Unfortunately, these techniques require switching to privileged mode for every peripheral I/O operation, which is unsuitable for LE-PLC systems requiring real-time performance. Meanwhile, hard-coded MMIO instrumentation also risks incomplete access control. ACES [16] divides a program into multiple isolated regions at compile time, reducing the extent of memory exposure under attack. However, due to over-privilege (one region can access unnecessary resources at runtime), LE-PLCs may still face security risks. MINION [31] introduces thread-level isolation by setting memory ranges for each thread through static analysis. Since many bare-metal embedded systems lack thread~~

abstractions, MINION's approach is difficult to apply directly to LE-PLCs. Targeting multi-core systems, uBOX [55] uses the MPU and non-privileged store instructions to build a lightweight sandbox mechanism supporting cross-core isolation at the same privilege level. However, it depends on the independence of an additional core, making it unsuitable for single-core LE-PLC systems. Its reliance on MPU bypass functionality also renders it incompatible with the ARMv8-M baseline architecture, further limiting its extensibility. Overall, these schemes each emphasize a specific security aspect and aim to mitigate partial threats at an acceptable runtime overhead. Yet most focus on only a single link in the attack chain. They possess inherent limitations and fail to comprehensively address the diverse security needs of broader applications.[old]

Other hardware-assisted mitigations reuse commodity embedded hardware rather than requiring full security extensions. EPOXY [17] and M2MON [30] identify privileged operations at compile time and mediate peripheral access by instrumentation. This strategy can protect sensitive hardware operations, but a PLC may execute MMIO operations in every scan cycle; forcing every peripheral operation through a privileged transition can increase scan-time jitter and delay actuator updates. ACES [16] partitions firmware into isolated compartments, thereby reducing memory exposure. Its compartment permissions, however, may still be over-privileged: a compartment can retain access to resources that are not needed for the specific runtime step being executed. In a PLC, such over-privilege is dangerous because unnecessary access to I/O images, set points, or actuator-related MMIO can directly influence the physical process. MINION [31] derives thread-level memory ranges through static analysis, but many bare-metal PLC runtimes do not expose threads as the isolation unit; the cyclic scan, not a general thread scheduler, is the dominant execution structure. uBOX [55] builds MPU-based isolation with a design that assumes a separate core and privileged code that can deliberately issue unprivileged memory accesses to bypass the active MPU context. This assumption is difficult to apply to single-core bare-metal PLCs, and the required MPU-bypass or unprivileged-access behavior is not part of the baseline execution model available on many ARMv8-M baseline devices. LiTEE-PLC differs from these mitigations by using the cyclic scan and I/O data dependencies as isolation hints. It does not mediate every MMIO operation uniformly; instead, it separates ordinary untrusted communication from the minimum trusted I/O path that affects physical actuation.[rev]

Software-based isolation and instrumentation. Software TEEs employ software-only methods to achieve secure isolation, eliminating the need for hardware modifications required by earlier approaches. Security MicroVisor (SV) [3] developed assembly-level instrumentation and software virtualization techniques, integrating secure virtual instruction into low-cost IoT devices through the toolchain, specifically targeting Harvard-based AVR architectures. PISTIS [24] extends this approach to more complex Von Neumann architectures, providing memory isolation, remote attestation, and secure updates. Both use full-stack instrumentation and runtime virtual instructions to achieve strong isolation between trusted modules and applications, while still allowing shared peripherals. Despite targeting low-end embedded devices, these software TEEs face two main limitations: 1) for two-world isolation and peripheral access control, they rely on extensive instrumentation of code in the untrusted world, causing every memory access behavior to be verified and resulting in high runtime overhead; 2) in network access control for LE-PLC input data from network sensor, they lack specific considerations for security and runtime overhead optimization. Given that network input is widely used in PLCs, this issue cannot be overlooked.[old]

Software-based isolation avoids requiring new security hardware by rewriting, instrumenting, or virtualizing untrusted code. Security MicroVisor (SV) [3] uses assembly-level instrumentation and software virtualization to introduce secure virtual instructions on low-cost devices. PISTIS [24] extends similar ideas to more complex Von Neumann architectures and supports memory isolation, remote attestation, and secure updates. These approaches preserve compatibility with legacy hardware, but their protection often depends on frequent checks on memory accesses, control transfers, or peripheral operations. They can also allow trusted and untrusted code to share peripherals by mediating accesses through software-defined interfaces. In this context, allowing shared peripherals means that the same physical device, such as a network interface, remains usable by both trusted and untrusted logic, while software checks determine whether each access is legal. This is useful for compatibility, but it does not by itself distinguish a safety-critical actuator command from non-critical protocol traffic unless the design incorporates PLC-specific I/O semantics.[rev]

The consequences are more severe for PLCs than for many general embedded devices. First, a PLC repeatedly executes a deterministic scan cycle, and even modest verification overhead can accumulate into scan-time jitter or missed actuation deadlines. Such timing deviations may not only reduce throughput, but also alter when outputs are applied to the physical process. Second, network inputs in a PLC are not ordinary application messages. They often correspond to remote sensor values or inter-controller signals that are copied into the input image and used by the control program to compute actuator outputs. If these inputs are forged, replayed, or delayed, the controller may make incorrect feedback decisions and drive the plant into unsafe states. If every network input is authenticated with expensive checks, the scan cycle may suffer unacceptable latency; if network inputs are not checked in a process-aware way, false-data injection may remain effective. LiTEE-PLC addresses this gap by combining MPU-enforced memory isolation with selective software checks: ordinary memory accesses are handled by MPU permissions, the minimum trusted I/O path is protected explicitly, and stronger source verification for network sensor data is triggered only when state-estimation deviations persist within a sliding window.[rev]

3 Background & Threat Model

3.1 Bare-metal Programmable Logic Controllers

As the core devices within control loops, LE-PLCs collect sensor data, execute control logic, manage actuator states, and respond to operator requests. LE-PLCs typically execute in a cyclic scanning mode that includes four steps: (1) During the input acquisition phase, sensor data is collected through physical pins and network buffers and updated to the LE-PLC's internal input buffer (a.k.a. the input image). (2) During the program execution phase, the LE-PLC performs logical operations, comparisons, counting, timing, and other functions based on the data stored in both input and output buffer (a.k.a. output image), then updates the results into the output buffer. (3) During the output updating phase, data from the output buffer is written simultaneously to physical pins and network buffers,

~~thereby actuating field devices (e.g. actuators, other controllers). (4) During the system communication phase, the LE-PLC prepares for the subsequent scan cycle by conducting operations such as system checks, clock resets, report transmission.~~[old]

As the core devices within control loops, bare-metal PLCs collect sensor data, execute control logic, update actuator states, and respond to operator or engineering requests. A typical PLC scan cycle contains four phases. In the input acquisition phase, sensor values from physical pins or network buffers are copied into the internal input image. In the program execution phase, the control program reads the input image and current output image, executes logic such as comparison, counting, timing, and arithmetic operations, and writes the results into the output image. In the output updating phase, the output image is committed to physical pins or network buffers and thereby affects actuators or downstream controllers. In the system communication phase, the PLC performs auxiliary tasks such as status reporting, clock maintenance, and preparation for the next scan cycle.[rev]

This cyclic structure is not only background knowledge, but also the motivation for LiTEE-PLC's methodology in Section 4. The scan cycle exposes a stable order between sensing, computation, actuation, and communication. It also makes the security relevance of each data object explicit: input images influence control decisions, output images influence actuation, and MMIO operations connect firmware states to physical devices. Therefore, LiTEE-PLC uses the scan semantics to identify which code and data must remain in the trusted domain, which communication components can be left in the untrusted domain, and which cross-domain interactions must be controlled. In particular, the fixed I/O path from input acquisition to output updating motivates the minimum trusted I/O path used later for selective peripheral protection.[rev]

The same structure also explains why generic embedded isolation is insufficient for PLCs. In many bare-metal PLCs, the runtime, protocol stack, control program, I/O images, and peripheral drivers are linked into a single firmware image and execute without process or kernel isolation. Once externally exposed communication code is compromised, the attacker may reach control data or actuator-related MMIO through the shared address space. At the same time, checking every memory or peripheral access would directly increase scan-cycle latency. These two observations lead to the design goal of Section 4: separate the trusted control path from exposed communication logic, while enforcing only the checks needed for memory, peripheral, and I/O-data interactions that can affect the physical process.[rev]

3.2 Memory Protection Unit

~~The MPU aims at providing hardware-based memory isolation capabilities and is widely supported in many embedded architectures [9–11, 23–11, 54–11], including ARM, MSP430, and RISCV. The MPU offers a set of hardware registers that, when configured, specifying memory regions with distinct access permissions for privileged and non-privileged modes, including read, write, and execute permissions.Any memory space not covered by these regions is referred to as the Background Region. Access permissions of the Background Region depends on the processor's running privilege level: privileged access is allowed by default, whereas non-privileged access triggers a MemManage exception. In addition, any memory access that violates the MPU configuration also triggers a MemManage exception. When configuring the system memory region's access permissions via the MPU, restrictions are imposed so that only privileged-level software may access that region. Any attempt by non-privileged-level software to do so will trigger a Bus fault, analogous to a MemManage exception.~~[old]

The MPU provides hardware-supported memory permission checks and is widely supported in embedded architectures [9–11, 23, 54], including ARM, MSP430, and RISC-V. An MPU configuration defines memory regions and assigns each region access permissions for privileged and non-privileged execution, such as read, write, and execute permissions. Memory not covered by an explicitly configured MPU region belongs to the background region. On typical ARM microcontrollers, privileged code can access the background region by default, whereas non-privileged access to an uncovered or disallowed region triggers a memory-management fault or a bus fault, depending on the accessed address range.[rev]

This mechanism suggests a lightweight way to construct isolation on bare-metal PLCs. Instead of instrumenting every load and store in software, LiTEE-PLC can place externally exposed code in non-privileged mode and use MPU regions to allow only the memory and peripheral ranges required by the untrusted domain. Trusted code, trusted data, I/O images, and safety-critical peripherals can be left outside the untrusted domain's accessible regions or mapped with stricter permissions. As a result, most illegal accesses are blocked by hardware faults rather than by frequent software checks. Section 4 builds on this observation to define the two-domain layout, the MPU permission policy, and the controlled software entry points needed for legitimate cross-domain calls.[rev]

However, the MPU is not sufficient for the complete PLC setting. First, an MPU region controls addresses rather than I/O semantics, so it cannot by itself decide whether a network transmission carries a safety-critical actuator command or non-critical protocol traffic. Second, shared peripherals may be needed by both trusted and untrusted logic, which means coarse-grained denial of all peripheral access would break normal PLC functions. Third, networked sensor data reaches the control program through the input image, so memory isolation cannot determine whether the received value is physically plausible. These limitations motivate the additional mechanisms introduced in Section 4: minimum-trusted-I/O-path protection for shared peripheral operations and state-estimation-based triggering for suspicious network inputs.[rev]

3.3 Design Challenges for MPU-Assisted PLC Isolation

MPU-assisted isolation for bare-metal PLCs faces three concrete challenges. The first challenge is domain partitioning: the control program, I/O images, setpoints, pin-mapping code, and safety-related drivers must be separated from externally exposed communication logic without breaking the PLC scan cycle. The second challenge is enforceable domain management: after partitioning, the system must translate the partition result into MPU regions, privilege-mode transitions, entry checks, and deployable binaries that can be reproduced by the build system. The third challenge is I/O-data-driven access control: PLCs often use shared communication peripherals and networked sensors, so address-level isolation alone cannot decide whether a data transfer carries actuator commands, ordinary protocol

traffic, or forged sensor values. Section 4 addresses these challenges in the same order.[rev]

For shared peripherals, the main difficulty is that security-critical and non-critical operations can traverse the same physical interface. A networked PLC may use the same Ethernet/SPI path to send actuator commands, receive sensor data, and exchange HMI or engineering messages. Moving the entire protocol stack into the trusted domain would enlarge the trusted code base, while checking every MMIO operation would increase scan-cycle latency. For network sensor inputs, the difficulty is different: a forged value may have a valid memory address and a valid packet format, but still be inconsistent with the physical process. Therefore, LiTEE-PLC uses a minimum trusted output path for actuator-related transmissions and a state-estimation-driven input check for network sensor data.[rev]

3.4 Threat Model

~~Our objective is to defend against all software-based attacks on the LE-PLC. Physical, hardware, and side-channel attacks fall outside the scope of this paper. We consider protection against physical attacks to be an orthogonal issue, with countermeasures including device deployment within physical shields or strict physical access control systems [47]. By assuming that the adversary has already possessed high-level privileges on other devices or network links that interact with the LE-PLC, we categorize the core threats to LE-PLCs as follows (see Figure ??) .~~[old]

Our objective is to defend against software-based attacks on the PLC firmware, memory, I/O data, and network-facing control path. Physical tampering, hardware modification, and side-channel attacks fall outside the scope of this paper. These threats are independent of the memory and peripheral isolation problem studied here: they require an attacker to physically access the device, alter hardware components, or extract information through physical leakage channels, whereas LiTEE-PLC focuses on restricting software-controlled memory, MMIO, and I/O-data flows after the device has booted. They are commonly addressed by deployment-level or hardware-level defenses, such as locked cabinets, tamper-resistant enclosures, controlled maintenance procedures, and physical access control systems [47]. Under this scope, we assume that the adversary may already control network links, external devices, or vulnerable software components interacting with the PLC, and we categorize the core software-level threats as follows (see Figure ??).[rev]

Assumptions. We make the following assumptions. The protected PLC platform provides an MPU that can enforce region-based access permissions for privileged and non-privileged execution. The PLC firmware source code and build process are available to the vendor, so that linker-script placement and compiler-assisted rewriting can be applied before deployment. The PLC vendor and firmware-signing infrastructure are trusted. Physical tampering, hardware replacement, and side-channel attacks are outside the scope of this work and are handled by deployment-level and hardware-level protections. Devices that exchange network data with the PLC, such as engineering stations, HMI stations, and sensors, can be provisioned with device identities, keys, sequence numbers, or other authentication metadata used by the access-control path.[rev]

4 LiTEE-PLC Methodology

~~The original Chapter 4 organized the methodology around assumptions, architecture description, memory isolation, and lightweight access control.~~[old]

LiTEE-PLC follows the three contributions stated in the Introduction. It first partitions the PLC firmware into a trusted domain and an untrusted domain. It then uses the MPU and compiler-generated metadata to manage the two domains at load time and runtime. Finally, it applies I/O-data-driven access control to the cross-domain paths that cannot be handled by address permissions alone. Figure 1 summarizes the three-stage methodology.[rev]

4.1 Partitioning Trusted and Untrusted Domains

Figure 2 shows the partitioning stage in detail. The partitioning stage takes the PLC firmware source, control program, I/O image definitions, peripheral map, and communication modules as input. It outputs two object sets. The trusted-domain set contains objects whose corruption can directly change sensing, control computation, or actuation. The untrusted-domain set contains externally exposed communication code and buffers whose compromise should not directly change the trusted control path. The overall role of this stage in the full LiTEE-PLC pipeline is shown in Figure 1.[rev]

LiTEE-PLC partitions firmware objects at function, data-object, and MMIO-region granularity. Trusted code includes the control program, scan-cycle scheduler, I/O-image update logic, pin-mapping routines, trusted update service, and entry handlers. Trusted data includes input images, output images, setpoints, actuator command variables, Kalman-filter state, device-identity metadata, and the counters used by the protected I/O path. Trusted MMIO includes physical I/O registers, chip-select registers on actuator-related paths, watchdog registers, and other safety-related peripheral registers. Untrusted code includes the network driver, TCP/IP or Modbus processing logic, engineering/HMI communication handlers, and other externally reachable parsers. Untrusted data includes the network stack state, protocol buffers, socket/session buffers, temporary packet fields, and non-critical communication variables.[rev]

The partitioning procedure is deterministic. First, LiTEE-PLC marks PLC control outputs, input/output images, setpoints, and safety-related MMIO symbols as trusted roots. Second, it marks network-facing parser functions, protocol handlers, network buffers, and engineering/HMI communication routines as untrusted exposure roots. Third, it resolves symbol references and call edges from these roots. Objects that are required to compute or commit actuator outputs are assigned to the trusted domain, while objects used only for receiving, parsing, or buffering external messages are assigned to the untrusted domain. Fourth, if an object is required by both domains, LiTEE-PLC does not duplicate the whole module; it keeps the large communication logic in the untrusted domain and exposes only a controlled entry or replay routine in the trusted domain.[rev]

4.2 MPU-Based Management of the Two Domains

Figure 3 summarizes how the partition result is converted into deployable and enforceable domains. The MPU-management stage has two phases. The deployment phase takes the trusted firmware

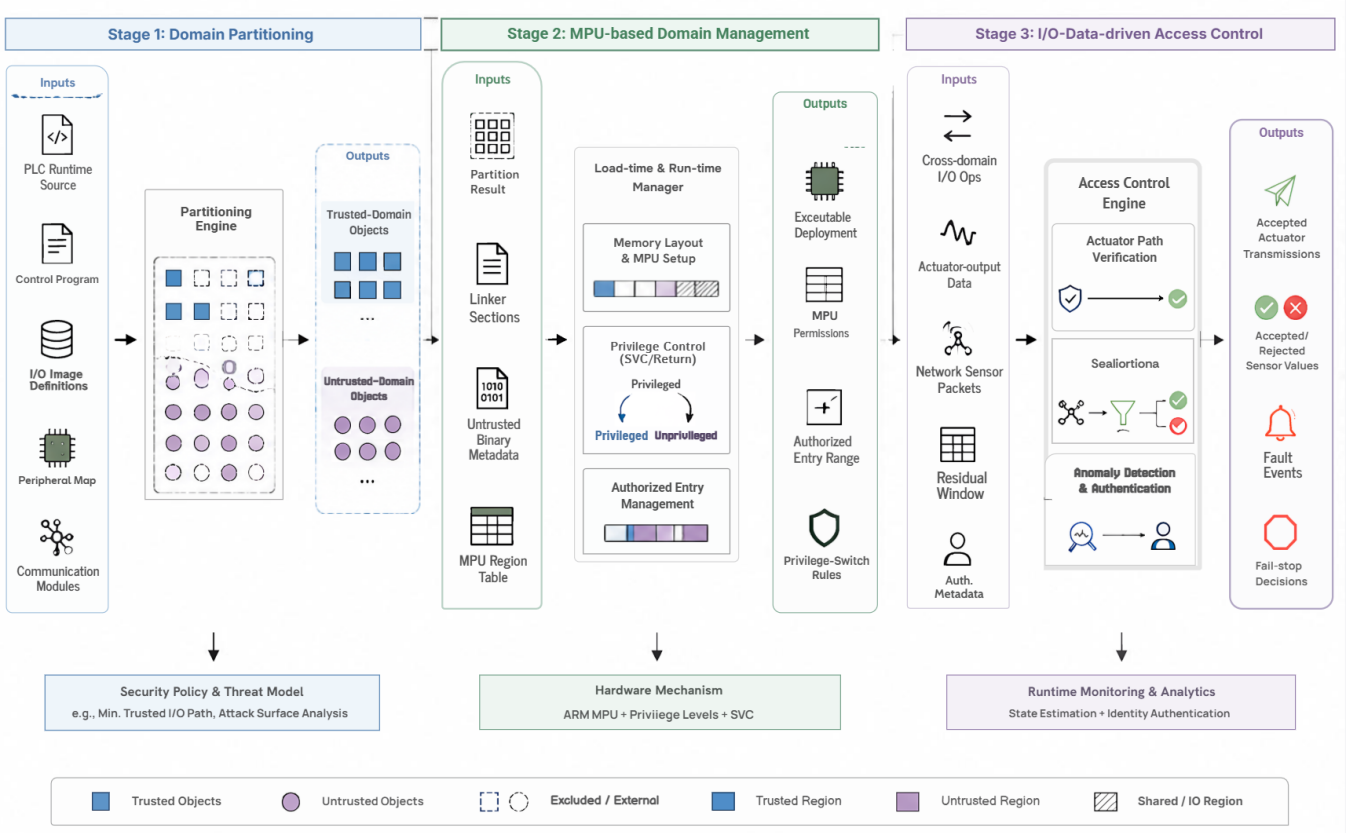

Figure 1: Overview of the LiTEE-PLC methodology. The framework first partitions PLC firmware into trusted and untrusted domains, then maps the partition result to MPU-managed regions and controlled entry points, and finally applies I/O-data-driven checks to shared output and network-input paths.

image, linker-generated section symbols, MPU policy table, loader, and untrusted-binary metadata as input. It outputs a memory image in which trusted and untrusted objects are placed at MPU-aligned addresses. The runtime phase takes this deployed image as input and enforces privilege separation, authorized entry invocation, and fault handling whenever the untrusted domain executes.[rev]

4.2.1 Deployment phase: configure, deploy, and transfer. The trusted deployment code configures MPU permissions in three steps. It reads linker symbols such as `__trusted_text_start`, `__trusted_data_end`, `__untrusted_text_start`, and `__svc_entry_start`; rounds each base and limit to the MPU alignment required by the target MCU; and writes the base address, size encoding, access-permission bits, execute-never bit, and enable bit into MPU registers. Trusted code and trusted data are either left in the privileged background region or mapped as privileged-only regions. Untrusted code and data are mapped with explicit non-privileged permissions. After all entries are written, the MPU is enabled before control is transferred to the untrusted domain.[rev]

The loader deploys the untrusted binary by parsing the secure-binary metadata generated by the compiler. The metadata contains the binary length, section count, section type, load address, entry offset, memory size, file offset, and permission flags. For each section, the loader checks that the destination address falls inside the pre-allocated untrusted region, copies the bytes from the binary payload, zero-initializes `.bss`-like areas, and rejects sections whose permissions do not match the MPU policy. The loader records the initial program counter, stack pointer, and authorized entry table after the copy finishes.[rev]

Control transfer is performed through a privilege-switch stub. Before entering untrusted code, the trusted runtime saves live registers, clears registers that may contain trusted values, installs the untrusted stack pointer, writes the untrusted entry address, and sets the processor to non-privileged mode. The untrusted domain can return to trusted services only through authorized SVC entries. On each SVC entry or fault, the handler validates the SVC number and the caller program counter against the authorized entry range before dispatching a trusted routine.[rev]

Table 2 shows how the seven effective non-background partitions are arranged. R0 and R1 separate untrusted code from untrusted writable data and prevent writable code or executable data. R2 stores packet data received from the network before verification. R3 exposes only registered communication ranges required by the untrusted stack. R4 contains SVC entry stubs, which lets the untrusted domain call trusted services only through fixed entry instructions. R5 provides read-only constants when required by both domains.

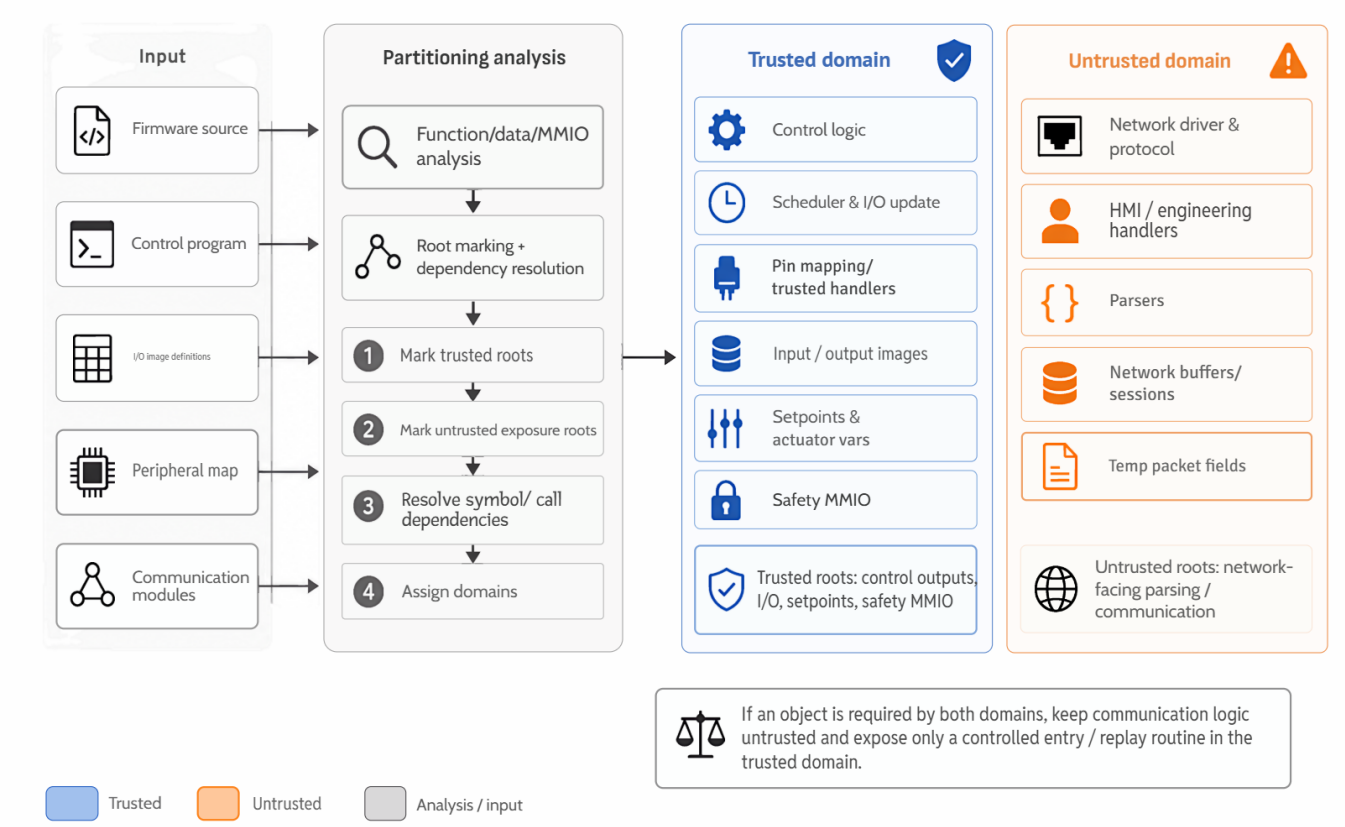

Figure 2: Partitioning trusted and untrusted domains. LiTEE-PLC takes the firmware source, control program, I/O-image definitions, peripheral map, and communication modules as inputs; marks trusted roots and untrusted exposure roots; resolves function, data-object, and MMIO dependencies; and outputs the trusted-domain and untrusted-domain object sets.

Table 2: MPU region layout for non-privileged execution.

Region	Mapped object	Permission	Purpose
R0	Untrusted code	RX, non-privileged	Execute communication code
R1	Untrusted data/stack/heap	RW, NX, non-privileged	Store protocol state and buffers
R2	Network buffer	RW, NX, non-privileged	Receive packet data
R3	Registered SPI/UART data path	RW, NX, non-privileged	Support allowed communication
R4	SVC entry stubs	RX, non-privileged	Enter trusted services at fixed points
R5	Read-only constants	R, NX, non-privileged	Share immutable tables
R6	Temporary shared window	RW or disabled	Enabled only for registered transfers
R7	Fault trampoline / reserved region	RX or disabled	Architecture-specific recovery path
Background	Trusted code/data/MMIO	Privileged only	Hide trusted objects from untrusted code

R6 is disabled by default and is enabled only when the loader or a trusted service needs a bounded shared transfer window. R7 is reserved for a fault trampoline or disabled on platforms that do not need it. All trusted code, trusted data, I/O images, setpoints, and protected MMIO ranges remain outside non-privileged access.[rev]

4.2.2 Compiler phase: software entries, interrupt hardening, and secure binary generation. The compiler phase turns the partition result into deployable code. It performs three transformations: adding software entry points, reinforcing interrupt security, and generating secure binaries with metadata. The compiler-phase portion of Figure 3 gives the code-level transformation view and the resulting artifacts used by deployment.[rev]

For software entry points, the compiler replaces each permitted cross-domain service call with a small SVC stub. The stub loads a service identifier into a fixed register, executes an SVC instruction, and passes only the arguments defined by the service ABI. The trusted SVC handler decodes the service identifier, checks whether the caller program counter belongs to an allowed call-site interval, and then dispatches the target trusted routine. The compiler emits the SVC stubs into a dedicated code section named `.entry`, so the MPU can expose only this entry area to non-privileged execution.[rev]

For interrupt hardening, the compiler first identifies ISR entry functions from the vector table. For each ISR, it builds a call graph and marks the functions reachable from that ISR. It then instruments the ISR prologue to store the interrupted context in a trusted stack area and clear registers that may contain trusted-domain values before untrusted code resumes. It also instruments memory reads, writes, and indirect jumps inside the ISR-reachable function set if they may access trusted ranges or jump outside the labeled target set. The ISR epilogue restores the saved context and returns to the interrupted privilege state.[rev]

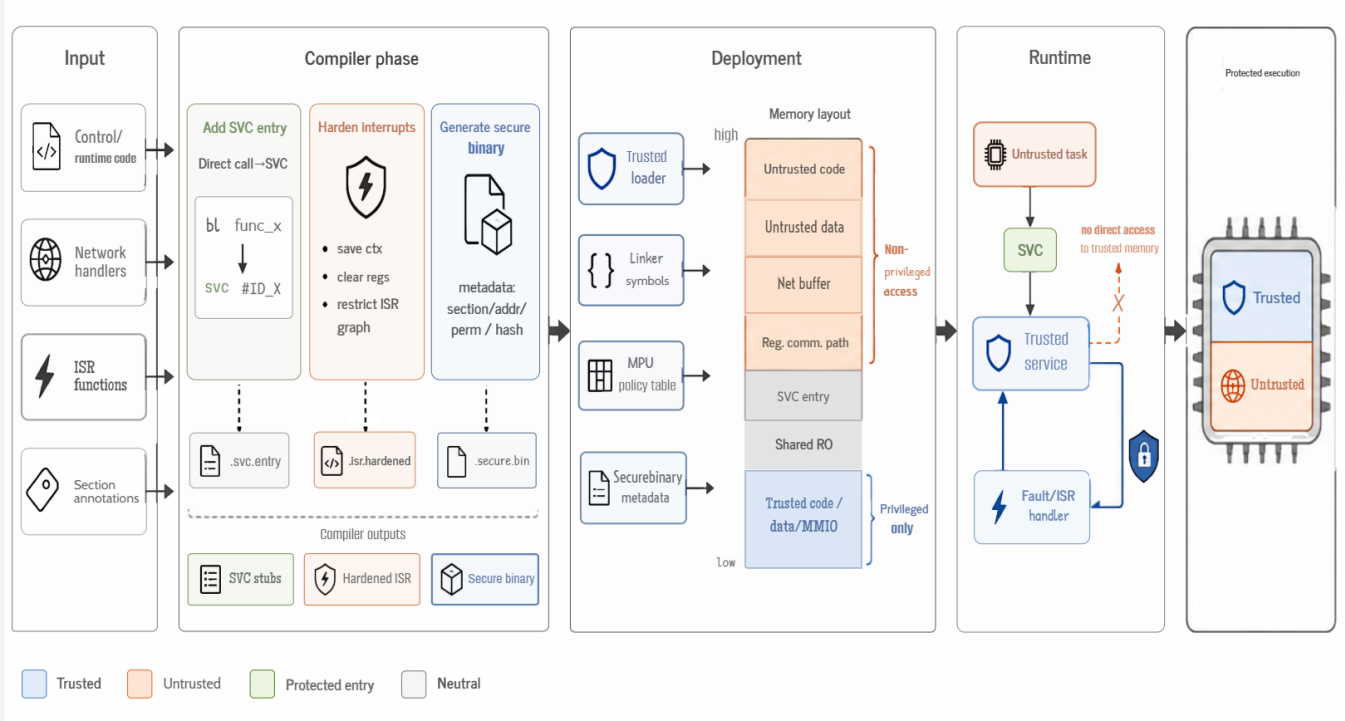

Figure 3: MPU-based management of the two domains. The compiler phase creates SVC entry stubs, hardened interrupt paths, and secure-binary metadata; the deployment phase uses the trusted loader, linker symbols, MPU policy table, and metadata to construct an MPU-aligned memory layout; and the runtime phase allows untrusted execution to enter trusted services only through checked SVC and fault-handling paths.

For secure-binary generation, the toolchain converts the untrusted ELF into a loader-readable image. The metadata header records the number of sections, section type, load address, file offset, memory size, permission flags, initial program counter, initial stack pointer, authorized SVC-entry range, and binary hash. The loader uses this metadata to verify that every section is loaded only into the untrusted memory range and that the declared permissions match the MPU policy. The final binary is encoded as `<total length, metadata header, section payloads>` so the deployment phase can reproduce the intended layout without relying on symbolic ELF processing on the microcontroller.[rev]

4.3 I/O-Data-Driven Access Control

Figure 4 summarizes the two I/O-data-driven mechanisms. I/O-data-driven access control handles the cross-domain operations that cannot be decided by address permissions alone. LiTEE-PLC applies two concrete mechanisms. For actuator-output data, it extracts a minimum trusted output path and executes that path through a protected SVC service. For network sensor inputs, it uses state estimation and a sliding-window detector to decide when a sensor value must be authenticated before it is copied into the trusted input image.[rev]

4.3.1 Minimum trusted output path for actuator data. The left half of Figure 4 shows the principle of the minimum trusted output path. The input to this mechanism is the set of actuator-related control variables produced by the control program. The output is a protected SVC service that sends only the corresponding actuator command through the shared communication peripheral. The rest of the network stack remains in the untrusted domain.[rev]

The forward slicing starts from manually annotated actuator-output roots, such as valve-opening percentage, motor enable bits, relay states, compressor speed commands, or emergency-shutdown commands. The LLVM pass follows use-def and def-use chains from these roots through assignments, pointer aliases, array stores, structure fields, function arguments, and return values. It also follows call edges from producers of actuator data to the send functions that consume this data. The slice stops when it reaches a peripheral write, a network send primitive, or a function boundary that does not affect the actuator value.[rev]

LiTEE-PLC distinguishes memory data from non-memory data during slicing. Memory data refers to values that are stored in addressable program objects, including global variables, static variables, stack slots whose address escapes, heap objects, arrays, structures, I/O-image fields, and packet-buffer fields. Non-memory data refers to temporary SSA values, constants, immediate operands, and register-only intermediate values that do not need persistent protection after the current instruction sequence. LiTEE-PLC protects memory data because attackers can revisit or overwrite it through memory accesses. Register-only values are instead protected by the entry, context-clearing, and interrupt-hardening rules.[rev]

The automatic extraction has four steps. The pass first collects annotated actuator-output roots and resolves their LLVM IR objects.

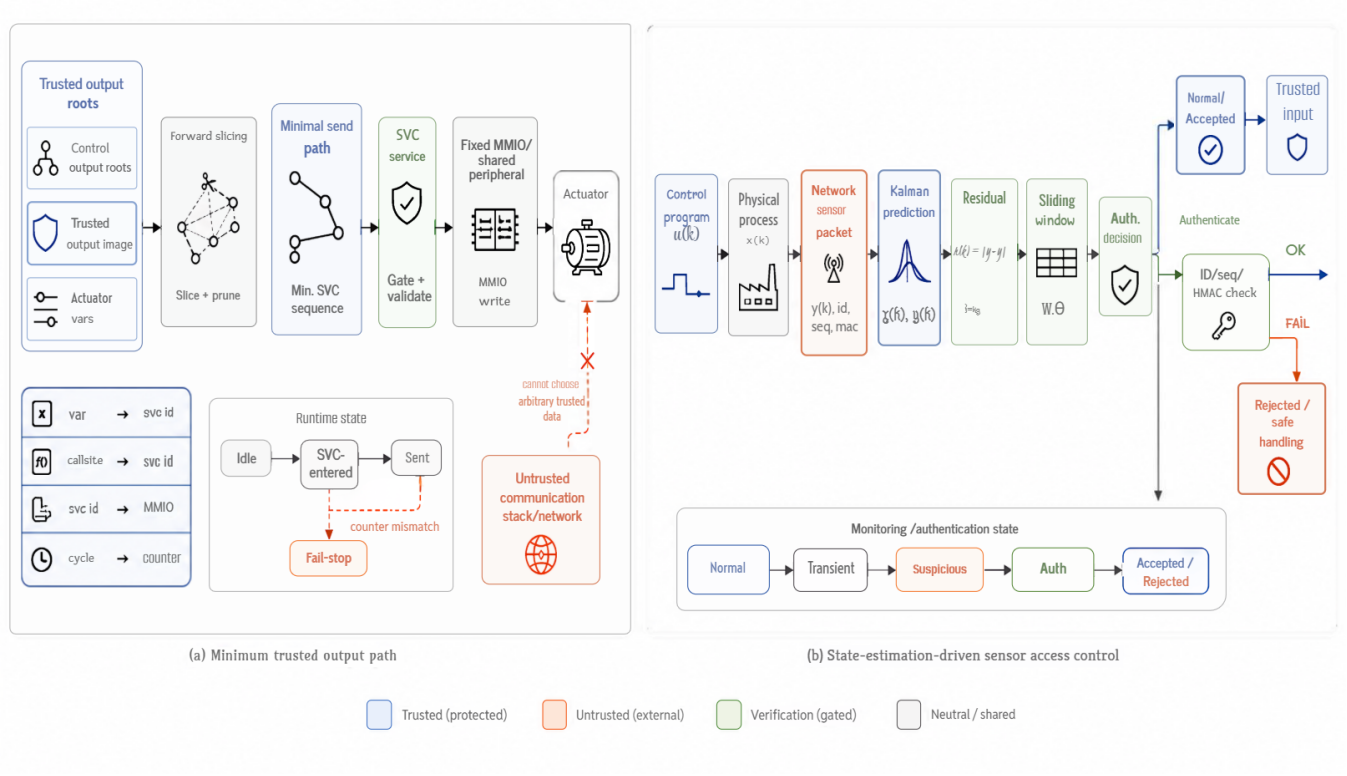

Figure 4: I/O-data-driven access control. The left half shows how LiTEE-PLC extracts and enforces the minimum trusted output path for actuator data, including SVC mapping and cycle-counter checking. The right half shows state-estimation-driven access control for network sensor data, where residual-window detection triggers authentication and only accepted values enter the trusted input image.

It then computes the forward slice until it reaches send-side sinks, such as packet serialization, SPI transmit-register writes, or network-send wrappers. Next, it prunes statements that do not affect the bytes transmitted to the actuator, such as logging, debug counters, or unrelated protocol parsing. Finally, it outlines the remaining statements into a new function whose parameters are limited to the trusted actuator data and the fixed peripheral/session descriptor. This outlined function becomes the body of the protected SVC service.[rev]

A minimum output path has the form `control variable -> output image -> serialization buffer -> chip-select/data register -> act` To outline this path, the compiler copies the necessary basic blocks into a new service routine, replaces external variables with explicit parameters or trusted loads, and removes calls that are not needed for the actuator command bytes. If the original path contains a shared helper function, the compiler clones only the helper statements that contribute to the actuator packet. The outlined routine is placed in the trusted text section, and its input variables are placed in the trusted data section.[rev]

The SVC service routine contains only the code needed to transmit the protected actuator data. It reads actuator values from the trusted output image, formats the protocol fields required by the target actuator, toggles the required chip-select line through trusted PIO operations, writes the data word or packet bytes to the transmit register or network send primitive, and updates a cycle-local execution counter. The untrusted domain cannot choose arbitrary trusted data or arbitrary chip-select states; it can only request a fixed SVC identifier at the call sites registered by the compiler.[rev]

Table 3: Mapping generated for the minimum trusted output path.

Before mapping	Generated map entry	After mapping	Runtime check
Actuator variable	variable → service ID	Trusted data load	Value read from tr
Send call site	caller PC → SVC ID	`svc #id` stub	Caller PC in allowe
Peripheral register	service ID → MMIO range	Trusted register write	MMIO range fixed
PLC scan cycle	cycle ID → counter	Monotonic counter update	Exactly once per cy

Table 3 shows how the map is built. Before mapping, the compiler observes variables, call sites, peripheral writes, and the scan-cycle boundary as separate program facts. It then creates map entries linking actuator variables to a service ID, call-site address ranges to the same service ID, service ID to a fixed peripheral range, and cycle number to a monotonic counter. After mapping, the untrusted path contains only SVC calls, while the trusted service loads protected data and performs fixed MMIO or send operations.[rev]

The runtime state machine for the protected output path contains four states: `Idle`, `SVC-Entered`, `Sent`, and `Fail-Stop`. At the beginning of each PLC scan cycle, the counter is reset and the state becomes `Idle`. When the authorized SVC is triggered from a registered call site, the state becomes `SVC-Entered`. After the service sends the

actuator data and updates the counter, the state becomes `Sent`. At the end of the scan cycle, the trusted runtime checks whether the expected counter value is present. If the expected value is missing or duplicated, the state changes to `Fail-Stop`. We use the term *counter mismatch* consistently for this failed check.[rev]

The high predictability of PLC control flow appears in the scan-cycle structure. In the evaluated PLC runtime, each cycle follows a fixed order: input-image update, control-program execution, output-image update, and communication/maintenance. The protected actuator transmission is expected after the output image has been computed and before the cycle finishes. LiTEE-PLC uses this cycle position as the trigger condition for the protected SVC path. Therefore, the check is not a generic heartbeat; it is a cycle-aligned verification that the actuator-output path was executed exactly once for the current control output.[rev]

The protected data are sent from the trusted output image to the actuator through the outlined service routine. For a serial or SPI-connected actuator, the service serializes the output value into protocol bytes, drives the chip-select line, writes the bytes to the transmit data register, and releases the chip-select line. For an Ethernet-connected actuator, the service prepares only the actuator-command payload and calls the registered send primitive without exposing the trusted output value to arbitrary untrusted code. In both cases, the source is the trusted output image and the destination is the actuator-side communication endpoint, not an arbitrary network buffer selected by the attacker.[rev]

4.3.2 State-estimation-driven control for network sensor data. The right half of Figure 4 shows the principle of network sensor access control. The mechanism connects the control program, physical process variables, network packets, and Kalman-filter states. The system state denotes the business-level physical process state, such as pressure, flow, temperature, valve position, or compressor state in the gas-transmission scenario. It does not refer to the PLC runtime state.[rev]

The control program and the physical process are connected by the state-space model. The control program computes a control input $u(k)$, such as valve opening or compressor command, from the input image and setpoints. The physical process evolves from $x(k)$ to $x(k+1)$ under $u(k)$ and process noise $w(k)$. Sensors report measured output $y(k)$ through network packets, and the measurement includes noise $v(k)$. Thus, $x(k)$ is the process state, $u(k)$ is the control output produced by the PLC logic, and $y(k)$ is the sensor value received through Modbus or another field protocol.[rev]

LiTEE-PLC uses the following linear or near-linear model for process variables that are stable around an operating point:[rev]

$$\begin{aligned} x(k+1) &= Ax(k) + Bu(k) + w(k), \\ y(k) &= Cx(k) + v(k). \end{aligned} \tag{1}$$

In this model, A, B, and C are obtained from process configuration, offline identification, or normal-operation traces. For gas transmission, $x(k)$ may include pressure and flow states, $u(k)$ may include valve and compressor commands, and $y(k)$ is the sensor value carried by network packets. Linear or near-linear modeling is therefore tied to the physical process controlled by the PLC, not to the PLC runtime or to the network protocol state.[rev]

Table 4: State machine for network sensor access control.

State	Condition	Action
Normal	$r(k) \le \lambda$	Copy value to trusted input image
Transient-Deviation	$r(k) > \lambda$, window count $\le \theta$	Monitor and keep local window state
Suspicious	window count $> \theta$	Trigger authentication
Authenticating	Auth metadata available	Verify device ID, sequence, and HMAC
Accepted	Authentication succeeds	Copy verified value to trusted input image
Rejected	Authentication fails	Reject value and keep safe input handling

The access-control state machine has six states: `Normal`, `Transient-Deviation`, `Suspicious`, `Authenticating`, `Accepted`, and `Rejected`. In `Normal`, residuals stay below the threshold and the received value is copied into the trusted input image. In `Transient-Deviation`, one or a few residuals exceed the threshold but the sliding-window condition is not yet satisfied, so the system keeps monitoring without triggering authentication. In `Suspicious`, the number of abnormal residuals in the window exceeds the threshold and authentication is triggered. In `Authenticating`, the system checks the sensor identity, sequence number, freshness metadata, and HMAC. A successful check enters `Accepted`; a failed check enters `Rejected` and the value is not copied into the trusted input image.[rev]

The threshold is $\lambda = k\sigma$, where σ is the standard deviation of residuals measured during normal operation and k is the threshold multiplier. The sliding window is a sequence of recent residual decisions for one sensor variable, not a window over arbitrary program variables. For each network sensor s, LiTEE-PLC maintains a window $W_s = \{b_{k-W+1}, \ldots, b_k\}$, where $b_i = 1$ if $r_s(i) > \lambda_s$ and $b_i = 0$ otherwise. The anomaly-count threshold θ is the minimum number of abnormal samples required to enter the `Suspicious` state. The parameters k, W, and θ are selected from normal-operation traces and validated in the evaluation by FAR/FNR and scan-cycle overhead.[rev]

The logical relation among threshold, attack, and authentication is as follows. A single packet is suspicious only if its residual exceeds λ. A sequence becomes an attack candidate only if enough suspicious packets appear inside the sliding window. Authentication is triggered only for such attack candidates. In packet terms, the input to authentication is not a user identity; it is a device/session identity for the sensor that produced the network value. The authenticated metadata includes sensor ID, key ID or session ID, sequence number, freshness counter or timestamp, measured value, and HMAC-256 over these fields. The result determines whether the value is copied into the trusted input image.[rev]

Short-term deviations can be self-corrected because many PLC-controlled processes contain feedback and inertia. For example, if one pressure sample briefly deviates because of noise, the next scan cycle reads a new value, the controller recomputes the actuator command, and the process may return toward the setpoint without changing authentication state. The sliding window filters this transient noise by requiring persistent residual violations before authentication. It reduces unnecessary authentication because isolated noise spikes do not cause expensive HMAC verification, while repeated or sustained forged values still cross the window threshold and are checked.[rev]

A sensitive setting in this mechanism consists of the residual threshold multiplier k, the residual standard deviation σ for each sensor variable, the window size W, the anomaly-count threshold θ, the sensor identity/key mapping, and the fallback action after

rejection. A smaller k or smaller θ makes detection more sensitive but may increase false alarms. A larger k or larger θ reduces false alarms but can delay detection because more abnormal samples are required before the state changes from `Transient-Deviation` to `Suspicious`. The delay is bounded by the window policy: in the worst case, authentication is triggered only after enough abnormal samples have accumulated within W scan cycles.[rev]

5 Evaluation

This section evaluates LiTEE-PLC from five aspects. Section 5.1 presents the experimental setup and integrates the implementation details of the prototype. Section 5.2 describes the attack scenarios and experimental design without analyzing results. Section 5.3 reports the security results, including whether attacks succeed before protection, whether they are blocked after enabling LiTEE-PLC, and whether normal control behavior is preserved. Section 5.4 evaluates system runtime overhead, scan-cycle overhead, and runtime comparison with representative prior mechanisms. Section 5.5 reports memory overhead. Discussion-oriented analysis that is not directly supported by experiment tables is moved to Section 6.[rev]

5.1 Experimental Setup

5.1.1 Prototype platform and runtime. We implement LiTEE-PLC on an Arduino Due board equipped with an ATSAM3X8E processor based on the ARM Cortex-M3 architecture, running at 84 MHz with 512 KB Flash and 96 KB SRAM. We omit the definition of a bare-metal system here because it has already been introduced in the Introduction and Background sections.[rev]

We use OpenPLC [12] as the PLC runtime. Commercial runtimes such as CODESYS [18] and Siemens PLC runtimes provide mature IEC 61131-3 execution environments, but their runtime core, protocol stack, and hardware abstraction layers are not fully accessible for low-level instrumentation, linker-script rewriting, MPU-region placement, and MMIO-level experiments. OpenPLC is representative for our purpose because it implements the same scan-cycle abstraction and IEC 61131-3-style control-program execution model while exposing all components that LiTEE-PLC must modify or measure: the runtime scheduler, I/O image mapping, Modbus communication path, protocol-stack boundary, generated control-program code, and hardware abstraction layer. This full controllability lets us precisely place trusted and untrusted objects, instrument entry points, replay peripheral operations, and measure scan-cycle overhead without treating the runtime as a black box.[rev]

The prototype integrates an Ethernet Shield 2 [5] through the SPI bus. We use this device because it represents a common microcontroller-class Ethernet deployment pattern: the Ethernet controller is attached as an external network peripheral, the MCU accesses it through MMIO/SPI operations, and the TCP/IP/Modbus stack must share the same peripheral path with control traffic. This structure is suitable for evaluating LiTEE-PLC because it creates exactly the shared-peripheral case studied in this paper: ordinary protocol processing should remain in the untrusted domain, while actuator-related traffic and minimum trusted I/O operations require controlled access.[rev]

5.1.2 Implementation details merged from the original implementation section. The LiTEE-PLC toolchain is built on the GCC 9.2 cross-compiler and contains a 102-line Makefile plus 581 lines of Python code for generating the untrusted binary and metadata. The runtime protection code contains 942 lines of C code and 360 assembly instructions. Lines of code (LOC) denotes non-comment, non-blank source lines. The 942 LOC consist of two parts: an 810-LOC secure-update service for firmware signature verification, decryption, and flashing, and a 132-LOC runtime isolation and monitoring core for MPU management, context switching, peripheral access control, and exception handling. The secure-update service is used only during firmware update and is not active in normal scan cycles, while the 132-LOC runtime core remains active during control execution.[rev]

For memory layout, GCC attributes and the linker script define the following sections: `.trusted.text` for trusted control code and runtime services, `.trusted.data` for input images, output images, setpoints, and other safety-critical data, `.untrusted.text` for externally exposed communication and protocol-processing code, `.untrusted.data` for the non-secure stack, heap, and network buffers, and `.svc.entry` for controlled cross-domain entry stubs. The linker script aligns these sections to MPU-region boundaries and emits section symbols such as `__trusted_text_start`, `__trusted_text_end`, `__untrusted_data_sta` and `__svc_entry_start`; the loader uses these symbols to configure the MPU and to verify that each entry address belongs to the authorized entry range.[rev]

For MPU configuration, the prototype maps untrusted code as read/execute but not writable, untrusted data as read/write but not executable, and the network-buffer region as read/write for communication logic. Trusted code, trusted data, I/O images, setpoints, SysTick, watchdog, and safety-critical MMIO regions are either placed in the privileged background region or mapped as privileged-only MPU regions. SPI and UART are configured as non-secure-accessible only where required for network communication, while chip-select operations and actuator-related data transfers are mediated by the minimum trusted I/O path. Any non-privileged access outside the allowed ranges triggers a MemManage fault or bus fault and is handled by the trusted exception path.[rev]

For peripheral replay, the untrusted network stack cannot directly toggle trusted chip-select operations for actuator-related transfers. Instead, when the untrusted code reaches a protected SPI operation, the SVC handler checks the call site and replays the minimal sequence on behalf of the trusted path. For example, `setSS` clears the PIO Clear Output Data Register (CODR) to drive the SPI chip-select line low, and `resetSS` sets the PIO Set Output Data Register (SODR) to release it. The MemManage fault handler similarly reconstructs the protected data word by merging the trusted data value with the chip-select/control bits and writes it to the SPI Transmit Data Register twice, matching the device protocol while preventing the untrusted domain from choosing arbitrary trusted data or chip-select states.[rev]

For network access control, the prototype maintains a Kalman filter for each networked sensor variable. During normal scan cycles, the PLC reads only the process-value registers required by the control program. When residuals persist in the sliding window, the PLC triggers an extended read that retrieves authentication material, including the HMAC-256 value, sequence number, and sensor identity fields. The verified value is copied into the trusted input-image slot only after the residual check and the optional authentication check succeed; otherwise the value is rejected or the controller enters a fail-safe handling path.[rev]

5.1.3 Testing platform and security-evaluation scenario. We use two complementary types of workloads in the evaluation. The first is a concrete gas-transmission pressure-regulation scenario, which is used for the attack-scenario and security-evaluation experiments. The second is a set of PLC benchmark workloads collected from public industrial-control projects, which is used for runtime-overhead and scalability evaluation. This separation avoids mixing a concrete business scenario with benchmark workloads: the gas-transmission scenario evaluates whether LiTEE-PLC preserves control safety under attacks, while the benchmark workloads evaluate whether LiTEE-PLC remains lightweight under different scan-cycle and slave-device scales.[rev]

The gas-transmission testing platform contains one LiTEE-PLC prototype, four networked sensor nodes, two actuator nodes, one engineering station, one HMI station, one attacker host, and one Ethernet router. All devices are connected to the same site network. Modbus TCP is used for process data exchange, ScadaBR [49] is used for HMI visualization and setpoint writes, and the engineering station is used for control-program download and firmware-update experiments. The attacker host is used only in attack-scenario experiments to inject malicious Modbus writes, replay forged sensor values, or deliver malicious control-program updates.[rev]

The concrete business process is gas-transmission pressure regulation. The PLC control program maintains downstream pipeline pressure and flow within safe ranges by reading upstream pressure, downstream pressure, gas flow rate, gas temperature, valve-position feedback, and compressor status. It then computes actuator commands for valve opening and compressor enable/speed control. The HMI displays pressure, flow, alarm state, and actuator state, and allows an operator to write pressure and flow setpoints. The engineering station deploys the ladder/structured-text control program and signed firmware-update package. This scenario is representative for security evaluation because it includes network sensor inputs, operator setpoints, actuator outputs, remote engineering operations, and shared Ethernet/SPI peripheral activity in a single control loop.[rev]

The transmitted data in the gas-transmission scenario are organized as follows. Sensor nodes send Modbus input-register values, including upstream pressure, downstream pressure, flow rate, temperature, valve-position feedback, compressor status, and sensor status. HMI writes use Modbus holding registers for pressure setpoint, flow setpoint, alarm-reset command, and manual/automatic mode switching. PLC-to-actuator messages contain valve-opening percentage, compressor enable bit, compressor speed setpoint, and emergency-shutdown command. During authenticated reads, sensor nodes additionally provide sensor identity, sequence number, timestamp or freshness counter, and HMAC-256 authentication data.[rev]

5.1.4 Runtime benchmark workloads. For runtime-overhead and scalability evaluation, we use benchmark PLC workloads rather than the single gas-transmission scenario alone. These workloads are collected from public industrial-control project examples [41] and ported to the LiTEE-PLC prototype. The final benchmark set should be reported explicitly rather than left for the reader to infer from the external repository. In our experiments, the benchmark set contains **TBD projects in total**, including **TBD** manufacturing projects, **TBD** energy projects, **TBD** chemical-processing projects, and **TBD** water-treatment or fluid-control projects. These projects are representative for performance evaluation because they exercise common PLC operations relevant to LiTEE-PLC: cyclic input-image update, setpoint-dependent control logic, actuator-output generation, Modbus communication, and peripheral I/O.[rev]

To evaluate scalability, we further configure four slave-device settings for these benchmark workloads. A slave is a networked field device, such as a sensor node or actuator node, that exchanges process data with the PLC. The 2-, 4-, 8-, and 16-slave settings represent increasingly complex control loops and correspond to maximum allowable scan-cycle times of 15 ms, 25 ms, 50 ms, and 100 ms, respectively. These settings are used only for runtime-overhead and scalability measurement, while the gas-transmission scenario is used for attack-scenario and security-evaluation experiments.[rev]

5.1.5 Compared systems and measurement method. We compare Baseline, LiTEE-PLC, M2MON, PISTIS, and a PLC-oriented TrustZone-based network-access-control design. This related-work configuration is part of the experimental setup rather than the result analysis. For fairness, all compared systems execute the same firmware, runtime, control programs, network inputs, and slave-device settings. For M2MON, all MMIO operations that can be statically identified are instrumented, and dynamically reached MMIO points are manually inspected and instrumented. For PISTIS, non-secure instructions are instrumented following its software-isolation strategy. For the PLC-oriented TrustZone-based design, the original per-network-input authentication policy is preserved; because Arduino Due does not provide TrustZone, only the memory/peripheral-isolation substrate is replaced by the LiTEE-PLC MPU-based substrate to allow comparison of the network-access-control policy under the same hardware platform.[rev]

5.2 Attack Scenarios

This subsection describes the experimental design of six attack scenarios listed in Table 5. The result analysis is reported in Section 5.3. The scenarios are derived from known PLC/ICS incidents and attack studies, including Stuxnet-style control-logic manipulation [14], Triton-style safety/control interference [38], PLC memory and firmware attacks [37, 40, 50, 52, 57], and false-data-injection attacks against feedback control [22]. The table gives the experimental target and expected process impact for all six attacks, while the following paragraphs explain the concrete attack code paths and the corresponding LiTEE-PLC protection mechanisms.[rev]

5.3 Security Evaluation

This subsection analyzes the security results summarized in Table 5. For each attack, we evaluate two questions: whether the unprotected baseline allows the attacker to corrupt control-relevant code, data, registers, or update paths, and whether LiTEE-PLC blocks the attack while preserving normal gas-transmission control behavior.[rev]

The unprotected baseline exposes multiple control-critical objects to compromised communication code or unauthorized engineering operations. Physical-pin hijacking forces the valve output and drives pressure from 1200 to 200 kPa and flow from 800 to 100 L/min. False-data injection corrupts the pressure/flow input path and makes the PLC read 50 kPa and 10 L/min. Watchdog-disable attacks remove reset-on-hang protection. Setpoint tampering changes the pressure setpoint from 1000 to 0 kPa. I/O-buffer tampering issues unsafe motor and valve outputs, and control-program injection corrupts

Table 5: Security evaluation results for the six attack scenarios in the gas-transmission experiment. "Loss before protection" reports the observed process-level impact under the unprotected baseline, while "After LiTEE-PLC" reports both the defense outcome and whether normal control behavior is preserved.

Attack scenario	Attack code / injected action	Targeted PLC object	Loss before protection	After LiTEE-PLC
Physical pin hijacking	Write `PIOB_SODR` at `0x400E1030`.	GPIO output pin for the valve.	Pressure drops from 1200 to 200 kPa, flow drops from 800 to 100 L/min, and the valve changes from 50% to 100%.	Blocked. `g_rejectCount=1` and an MPU fault is triggered. The process remains at pressure 1200 kPa, flow 800 L/min, and valve 50%.
False data injection	Write `int_input[0..1]`; Kalman filter and sliding-window check with threshold 9.647 and window 5.	Pressure/flow sensor input buffer.	Pressure changes from 1200 to 50 kPa and flow changes from 800 to 10 L/min. The PLC reads corrupted data.	Detected by the Kalman filter. `g_rejectCount=2`; residuals exceed 9.647 for five consecutive samples, the window triggers, anomalous values are rejected, and pressure/flow stay at 1200 kPa and 800 L/min.
Watchdog disable	Write `WDT_MR` at `0x400E1454`.	Watchdog timer control register.	The watchdog is disabled and the system loses reset-on-hang protection.	Blocked. `g_rejectCount=3` and an MPU fault is triggered. The watchdog remains active.
Setpoint tampering	Write `int_output[0]`.	Pressure setpoint register.	The pressure setpoint changes from 1000 to 0 kPa, causing the PLC to target zero pressure.	Rejected. `g_rejectCount=4` and HMAC verification fails. The pressure setpoint remains 1000 kPa.
I/O-buffer tampering	Write `bool_output[0][0..1]`.	Output-image memory for motor/valve control.	The motor command changes from 1500 to 3000 RPM and the valve changes from 50% to 0%.	Blocked. `g_rejectCount=5` and an MPU fault is triggered. The motor remains at 1500 RPM and the valve remains at 50%.
Control-program injection	Write Flash address `0x90000`.	Program-code region in Flash.	The program code is corrupted and the program hash becomes invalid.	Blocked. `g_rejectCount=6`; Flash Region 2 is read-only, HMAC verification remains valid, and the original program is preserved.

the Flash program region. These attacks affect different parts of the PLC execution path, but they all attempt to change the physical process by corrupting control inputs, outputs, parameters, or program code.[rev]

After enabling LiTEE-PLC, all six attacks are blocked before they can affect the trusted control path. Unauthorized MMIO and memory writes trigger MPU faults, forged sensor values are rejected by the Kalman-filter residual and sliding-window detector, unauthorized setpoint writes fail HMAC verification, and the modified control program is not installed into the trusted Flash region. Across the six attacks, `g_rejectCount` increases from 1 to 6, matching the six blocked or rejected attempts. The protected system therefore preserves the legitimate sensing-control-actuation workflow: pressure and flow remain at 1200 kPa and 800 L/min under the relevant attacks, the valve remains at 50%, the motor remains at 1500 RPM, the watchdog remains active, and the original control program is preserved.[rev]

5.3.1 Network access control and parameter selection. The background explanation of residual thresholding, the 3σ rule, and sliding-window robustness is moved to Section 5.1. This subsection focuses on the parameter-sweep results. Each configuration is evaluated with 100 normal cycles for FAR and 50 attack cycles for FNR. Runtime is measured with a microsecond-resolution timer, so the reported 13 μs value should be interpreted as the measured upper-bound value at this timer resolution.[rev]

Table 6 shows that all evaluated parameter settings detect the injected false-data attacks in this experiment, yielding 0% FNR. The main difference is the false-alarm behavior under normal cycles. Increasing the residual multiplier k reduces FAR from 54.0% at $k = 1$ to 0% at $k = 4$ and $k = 5$. Increasing the window size also lowers FAR, from 14.0% at $w = 1$ to 0% at $w = 20$, but a larger window can delay authentication because more consecutive abnormal samples are required before the state changes.[rev]

Table 6: Security and runtime trade-off for network access control. Each parameter setting is evaluated with 100 normal cycles for FAR and 50 attack cycles for FNR. Runtime is measured with `micros()` and stays at 13 μs for all tested settings.

Parameter	Value	FAR / FNR	NAC runtime
k	1	54.0% / 0%	13 μs
k	2	31.0% / 0%	13 μs
k	3	9.0% / 0%	13 μs
k	4	0% / 0%	13 μs
k	5	0% / 0%	13 μs
w	1	14.0% / 0%	13 μs
w	3	11.0% / 0%	13 μs
w	5	9.0% / 0%	13 μs
w	10	4.0% / 0%	13 μs
w	20	0% / 0%	13 μs

We therefore select $k = 4$ and $w = 5$ as the default network-access-control setting. This setting achieves 0% FAR and 0% FNR in the parameter sweep while keeping the measured NAC runtime unchanged at 13 μs. The conventional 3σ setting, $k = 3$ and $w = 5$, also keeps FNR at 0%, but it produces a 9.0% FAR in the normal-cycle measurement and would trigger unnecessary authentication more often.[rev]

5.4 Runtime Overhead

This subsection separates three runtime questions that were mixed in the original text. First, we report the one-time system startup overhead of LiTEE-PLC. Second, we evaluate the scan-cycle runtime overhead introduced during normal operation. Third, we compare LiTEE-PLC with prior mechanisms under the same workload and slave-device settings. The related-work configuration has already been described in Section 5.1; this subsection discusses only measured data.[rev]

LiTEE-PLC has two startup costs: MPU-policy configuration costs 40 μs on average, and non-secure program deployment costs 187 ms

on average. Because both costs occur only once at startup, they are reported as system runtime overhead but excluded from the scan-cycle overhead measured during normal operation.[rev]

As shown in the "Time limit" row of Table 7, all slave settings have explicit timing budgets. The "LiTEE-PLC cycle time" row shows that LiTEE-PLC stays within all budgets and adds only 0.98% overhead on average. The component-overhead rows further show that both peripheral access control and network access control are measured directly, avoiding a split explanation across different subsections.[rev]

5.5 Memory Overhead

LiTEE-PLC introduces 11,640 bytes of additional storage usage, including 2,276 bytes of SRAM and 9,364 bytes of Flash. This corresponds to 2.31% of the 96 KB SRAM and 1.82% of the 512 KB Flash available on Arduino Due. Table 8 reports the breakdown and includes related systems when their storage data are available. TBD indicates that the corresponding paper or implementation artifact does not provide a directly comparable value yet.[rev]

The code-size and module-function discussion previously placed under TCB analysis is moved here and into Section 5.1. Instead of using a separate subsection with only one paragraph, we report the code-size facts as part of the implementation and memory-overhead setup. The runtime core is 132 lines of code (LOC), while the secure-update service is 810 LOC and is inactive during normal scan cycles.[rev]

6 Discussion & Future Work

Despite LiTEE-PLC's advantages in security and real-time performance, there remains scope for further improvements:

Driver Code Minimization. Minimizing driver code in the secure world reduces defects and simplifies verification and manual auditing. We recommend methods from[26] and[51], which record and replay driver-device interactions to generate a minimal functional "driverlet". Although orthogonal to our approach, these methods and our proposed mechanism can complement each other to enhance TEE security and trustworthiness.

MPU Constraints. Since the Cortex-M3 processor offers only eight MPU regions (Regions 0–7), the number of enabled non-secure peripherals is limited to six regions at most (with Regions 0–1 used for code/data protection). In extreme scenarios, this may not suffice. Additionally, System memory regions (as detailed in Section 2.2) encompass critical core peripherals[56]such as the Nested Vectored Interrupt Controller (NVIC) and SysTick timer. Should the non-secure world require access to these components, Bus exceptions similar to MemManage exceptions would be triggered (though our experimental targets do not involve such access patterns). For both aforementioned peripheral types, we recommend adopting the same peripheral access control mechanism described in Section 4.4.1, where operations are uniformly proxied through exception handlers in the secure world. Experiments confirm that μs-level overhead remains predictably low. Notably, the region quantity limitation would be significantly mitigated in processors supporting more MPU regions, such as Cortex-M7[8], PowerPC e200[21], and certain RISC-V implementations[4].

7 Conclusion

This paper introduces LiTEE-PLC, a lightweight trusted execution environment specifically designed for LE-PLC. By employing fine-grained decoupling of LE-PLC software modules and leveraging MPUs, LiTEE-PLC effectively isolates core modules from potential security threats without requiring any hardware modifications. Through lightweight peripheral access control based on the Minimum Trusted I/O Path and lightweight network access control based on state estimation, LiTEE-PLC further ensures complete, low-overhead, and highly secure cross-world resource interactions. Our evaluation shows that LiTEE-PLC maintains real-time performance with negligible runtime and memory overhead, making it an ideal solution for resource-constrained industrial environments.

References

[1] Esmerald Aliaj, Ivan De Oliveira Nunes, and Gene Tsudik. 2022. {GAROTA}: generalized active {Root-Of-Trust} architecture (for tiny embedded devices). In *31st USENIX Security Symposium (USENIX Security 22)*. 2243–2260.

[2] Ephrem Ryan Alphonsus and Mohammad Omar Abdullah. 2016. A review on the applications of programmable logic controllers (PLCs). *Renewable and Sustainable Energy Reviews* 60 (2016), 1185–1205.

[3] Mahmoud Ammar, Bruno Crispo, Bart Jacobs, Danny Hughes, and Wilfried Daniels. 2019. SμV—The Security MicroVisor: A Formally-Verified Software-Based Security Architecture for the Internet of Things. *IEEE Transactions on Dependable and Secure Computing* 16, 5 (2019), 885–901.

[4] Andes Technology Corporation. 2024. AndesCore A25 – RISC-V Superscalar Processor with Linux Support. https://www.andestech.com/en/products-solutions/andescore-processors/riscv-a25/. Accessed: 2025-04-14.

[5] Arduino. 2025. Arduino Ethernet Shield 2. https://store.arduino.cc/products/arduino-ethernet-shield-2. Accessed: 2025-04-15.

[6] ARM. 2011. *TrustZone technology for Armv8-A.* https://developer.arm.com/ip-products/security-ip/trustzone/trustzone-for-cortex-a#armv8-a [Online; accessed 7-April-2025].

[7] ARM. 2015. *TrustZone technology for Armv8-M.* https://developer.arm.com/ip-products/security-ip/trustzone/trustzone-for-cortex-m [Online; accessed 7-April-2025].

[8] ARM Limited. 2010. *ARMv7-M Architecture Reference Manual.* ARM Limited.

[9] ARM Limited. 2010. *Cortex-M4 Technical Reference Manual.* ARM Limited.

[10] ARM Limited. 2011. *Cortex-R4 and Cortex-R4F Technical Reference Manual.* ARM Limited.

[11] ARM Limited. 2016. *ARMv8-M Architecture Reference Manual.* ARM Limited.

[12] Autonomy Logic. 2025. Autonomy Logic. https://autonomylogic.com/. Accessed: 2025-04-15.

[13] Ahmad Ammar Nor Azlin, Hasmah Mansor, Ahmad Zawawi Hashim, and Teddy Surya Gunawan. 2017. Development of modular smart farm system. In *2017 IEEE 4th International Conference on Smart Instrumentation, Measurement and Application (ICSIMA)*. IEEE, 1–6.

[14] Marie Baezner and Patrice Robin. 2017. *Stuxnet.* Technical Report. ETH Zurich.

[15] Ferdinand Brasser, Brahim El Mahjoub, Ahmad-Reza Sadeghi, Christian Wachsmann, and Patrick Koeberl. 2015. TyTAN: Tiny trust anchor for tiny devices. In *Proceedings of the 52nd annual design automation conference*. 1–6.

[16] Abraham A Clements, Naif Saleh Almakhdhub, Saurabh Bagchi, and Mathias Payer. 2018. ACES: Automatic compartments for embedded systems. In *27th USENIX Security Symposium (USENIX Security 18)*. 65–82.

[17] Abraham A Clements, Naif Saleh Almakhdhub, Khaled S Saab, Prashast Srivastava, Jinkyu Koo, Saurabh Bagchi, and Mathias Payer. 2017. Protecting bare-metal embedded systems with privilege overlays. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 289–303.

[18] CODESYS GmbH. 2025. CODESYS - the IEC 61131-3 Automation Software. https://www.codesys.com/. Accessed: 2025-04-15.

[19] Victor Costan and Srinivas Devadas. 2016. Intel SGX explained. *Cryptology ePrint Archive* (2016).

[20] Oliver Eigner, Philipp Kreimel, and Paul Tavolato. 2016. Detection of man-in-the-middle attacks on industrial control networks. In *2016 International Conference on Software Security and Assurance (ICSSA)*. IEEE, 64–69.

[21] Freescale Semiconductor. 2012. *MPC5674F Microcontroller Reference Manual.* Freescale Semiconductor. Available online: https://www.nxp.com/docs/en/reference-manual/MPC5674FRM.pdf.

[22] Serkan Gönen, H Hüseyin Sayan, Ercan Nurcan Yılmaz, Furkan Üstünsoy, and Gökçe Karacayılmaz. 2020. False data injection attacks and the insider threat in smart systems. *Computers & Security* 97 (2020), 101955.

Table 7: Scan-cycle runtime under different slave-device settings. A slave is a networked field device, such as a sensor or actuator node, that exchanges process data with the PLC. The 2-, 4-, 8-, and 16-slave settings correspond to increasingly complex control loops and therefore use different scan-cycle time limits. Component-overhead rows expose LiTEE-PLC's two runtime modules: peripheral access control and network access control.

Metric	2 slaves	4 slaves	8 slaves	16 slaves	Average	Note
Scenario	Manufacturing	Energy	Chemical/process	Water/gas-fluid	–	Workload class
Time limit	15 ms	25 ms	50 ms	110 ms	–	Max allowed scan time
Baseline cycle time (min/avg/max)	2.94 / 3.21 / 4.70	4.85 / 5.54 / 7.90	9.67 / 11.42 / 16.83	20.26 / 23.20 / 28.71	–	ms
M2MON cycle time (min/avg/max)	3.51 / 3.86 / 5.52	5.98 / 6.69 / 9.71	10.12 / 13.84 / 18.90	22.87 / 27.90 / 33.74	–	ms
M2MON overhead vs. baseline avg.	+20.1%	+20.8%	+21.2%	+20.3%	+20.6%	Runtime overhead
M2MON peripheral-access overhead	0.68 ms	0.98 ms	2.05 ms	4.03 ms	–	Main module overhead
PISTIS cycle time (min/avg/max)	5.08 / 5.85 / 6.96	9.24 / 10.07 / 13.23	19.73 / 21.23 / 25.98	38.11 / 42.27 / 48.10	–	ms
PISTIS overhead vs. baseline avg.	+82.4%	+81.7%	+85.9%	+83.1%	+83.3%	Runtime overhead
PISTIS memory-isolation overhead	2.92 ms	4.51 ms	9.78 ms	19.42 ms	–	Main module overhead
TrustZone-based NAC cycle time (min/avg/max)	5.28 / 5.95 / 7.20	10.37 / 10.24 / 13.80	20.31 / 21.20 / 26.40	36.82 / 43.19 / 44.73	–	ms
TrustZone-based NAC overhead	+85.3%	+84.8%	+85.6%	+86.1%	+85.5%	Runtime overhead
TrustZone-based network-access overhead	2.47 ms	4.21 ms	8.07 ms	15.84 ms	–	Main module overhead
LiTEE-PLC cycle time (min/avg/max)	2.96 / 3.24 / 4.83	4.87 / 5.60 / 8.37	9.70 / 11.50 / 17.62	20.30 / 23.48 / 29.98	–	ms
LiTEE-PLC overhead vs. baseline avg.	+0.9%	+1.1%	+0.7%	+1.2%	+0.98%	Runtime overhead
LiTEE memory-isolation overhead	12.64 μs	23.93 μs	50.27 μs	97.80 μs	–	Module overhead
LiTEE peripheral-access overhead	13.27 μs	24.91 μs	48.89 μs	102.53 μs	–	Module overhead
LiTEE network-access overhead	19.71 μs	37.14 μs	79.42 μs	153.30 μs	–	Module overhead

Table 8: Memory overhead comparison. "TBD" means that the corresponding related work does not report a directly comparable value in the available artifact yet.

System / component	SRAM overhead	Flash overhead
M2MON	TBD	TBD
PISTIS	TBD	TBD
PLC-oriented TrustZone-based NAC	TBD	TBD
LiTEE-PLC memory isolation	1,042 bytes	
LiTEE-PLC peripheral access control	114 bytes	
LiTEE-PLC network access control	1,132 bytes	
LiTEE-PLC security services	3,256 bytes	
MPU alignment fragmentation	144 bytes	~3.8 KB
LiTEE-PLC total	2,276 bytes	9,364 bytes
Device capacity share	2.31%	1.82%

[23] Michele Grisafi, Mahmoud Ammar, and Bruno Crispo. 2022. On the (in) security of memory protection units: A cautionary note. In *2022 IEEE International Conference on Cyber Security and Resilience (CSR)*. IEEE, 157–162.

[24] Michele Grisafi, Mahmoud Ammar, Marco Roveri, and Bruno Crispo. 2022. {PISTIS}: Trusted computing architecture for low-end embedded systems. In *31st USENIX Security Symposium (USENIX Security 22)*. 3843–3860.

[25] Harsha Priyanka Guntaka, Rajesh Kushalkar, Nirmala Venkat, Akshay Chipkar, Vishnu Easwaran, and Kannan Moudgalya. 2022. Modular Hardware, Open Source Software and Training Material for PLC Training. In *2022 10th International Conference on Control, Mechatronics and Automation (ICCMA)*. IEEE, 237–242.

[26] Liwei Guo and Felix Xiaozhu Lin. 2022. Minimum viable device drivers for arm trustzone. In *Proceedings of the Seventeenth European Conference on Computer Systems*. 300–316.

[27] S Gupta and P Bhowal. 2004. Simplified open-loop anti-sway technique. In *Proceedings of the IEEE INDICON 2004. First India Annual Conference, 2004.* IEEE, 225–228.

[28] Yanjun Hou, Yongxing Hao, and Wenzhuo Yuan. 2011. Design and implementation of PLC multimedia software. In *2011 International Conference on Multimedia Technology*. IEEE, 3475–3477.

[29] David Kaplan, Jeremy Powell, and Tom Woller. 2016. AMD memory encryption. *White paper* 13 (2016), 12.

[30] Arslan Khan, Hyungsub Kim, Byoungyoung Lee, Dongyan Xu, Antonio Bianchi, and Dave Jing Tian. 2021. {M2MON}: Building an {MMIO-based} security reference monitor for unmanned vehicles. In *30th USENIX Security Symposium (USENIX Security 21)*. 285–302.

[31] Chung Hwan Kim, Taegyu Kim, Hongjun Choi, Zhongshu Gu, Byoungyoung Lee, Xiangyu Zhang, and Dongyan Xu. 2018. Securing Real-Time Microcontroller Systems through Customized Memory View Switching.. In *NDSS*.

[32] Patrick Koeberl, Steffen Schulz, Ahmad-Reza Sadeghi, and Vijay Varadharajan. 2014. TrustLite: A security architecture for tiny embedded devices. In *Proceedings of the Ninth European Conference on Computer Systems*. 1–14.

[33] Enas Dhuhri Kusuma, Nifwan Arbi Nugroho, Addin Suwastono, and Sujoko Sumaryono. 2024. Perbandingan Performa OpenPLC dengan PLC Komersial. *Journal of Applied Electrical Engineering* 8, 2 (2024), 125–131.

[34] Hongda Li, Feng Wei, and Hongxin Hu. 2019. Enabling dynamic network access control with anomaly-based IDS and SDN. In *Proceedings of the ACM international workshop on security in software defined networks & network function virtualization*. 13–16.

[35] Mengyuan Li, Yuheng Yang, Guoxing Chen, Mengjia Yan, and Yinqian Zhang. 2024. Sok: Understanding design choices and pitfalls of trusted execution environments. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. 1600–1616.

[36] Zhiang Li, Daisuke Mashima, Wen Shei Ong, Ertem Esiner, Zbigniew Kalbarczyk, and Ee-Chien Chang. 2024. On Practicality of Using ARM TrustZone Trusted Execution Environment for Securing Programmable Logic Controllers. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. 947–961.

[37] Efrén López-Morales, Ulysse Planta, Carlos Rubio-Medrano, Ali Abbasi, and Alvaro A Cardenas. 2024. {SoK}: Security of Programmable Logic Controllers. In *33rd USENIX Security Symposium (USENIX Security 24)*. 7103–7122.

[38] William B McKinnon and Randolph L Kirk. 2014. Triton. In *Encyclopedia of the solar system*. Elsevier, 861–881.

[39] Luis I Minchala, Jonnathan Peralta, Paul Mata-Quevedo, and Jaime Rojas. 2020. An approach to industrial automation based on low-cost embedded platforms and open software. *Applied Sciences* 10, 14 (2020), 4696.

[40] Tanmaya Mishra, Thidapat Chantem, and Ryan Gerdes. 2022. Survey of control-flow integrity techniques for real-time embedded systems. *ACM Transactions on Embedded Computing Systems (TECS)* 21, 4 (2022), 1–32.

[41] MrPLC. [n. d.]. Industrial Automation Control Systems Community. https://mrplc.com/. Accessed: 2025-04-15.

[42] Antonio Muñoz, Ruben Ríos, Rodrigo Román, and Javier López. 2023. A survey on the (in) security of trusted execution environments. *Computers & Security* 129 (2023), 103180.

[43] Kwaw Tawiah Ndede. 2022. A low-cost microcontroller-based PLC for industrial automation of a gold processing plant. (2022).

[44] Saulius Niauronis. 2023. OPENPLC Hardware Speed Performance Comparison. *PROFESSIONAL STUDIES: Theory And Practice* 27, 1 (2023), 65–71.

[45] Job Noorman, Pieter Agten, Wilfried Daniels, Raoul Strackx, Anthony Van Herrewege, Christophe Huygens, Bart Preneel, Ingrid Verbauwhede, and Frank Piessens. 2013. Sancus: Low-cost trustworthy extensible networked devices with a zero-software trusted computing base. In *22nd USENIX Security Symposium (USENIX Security 13)*. 479–498.

[46] Arttu Paju, Muhammad Owais Javed, Juha Nurmi, Juha Savimäki, Brian McGillion, and Billy Bob Brumley. 2023. Sok: A systematic review of tee usage for developing trusted applications. In *Proceedings of the 18th International Conference on Availability, Reliability and Security*. 1–15.

[47] Srivaths Ravi, Anand Raghunathan, and Srimat Chakradhar. 2004. Tamper resistance mechanisms for secure embedded systems. In *17th International Conference on VLSI Design. Proceedings.* IEEE, 605–611.

[48] Rajashree Rout, Sudhansu Kumar Samal, and Gautam Modak. 2024. Design of Low-Cost PLC to Drive Three-Phase Induction Motor using Variable Frequency Drive. In *2024 Control Instrumentation System Conference (CISCON).* IEEE, 1–5.

[49] ScadaBR Community. n.d.. ScadaBR: Open Source SCADA System. https://scadabr.org/. Accessed: 2025-04-15.

[50] Ruimin Sun, Alejandro Mera, Long Lu, and David Choffnes. 2021. SoK: Attacks on industrial control logic and formal verification-based defenses. In *2021 IEEE European Symposium on Security and Privacy (EuroS&P).* IEEE, 385–402.

[51] Jinwen Wang, Ao Li, Haoran Li, Chenyang Lu, and Ning Zhang. 2022. Rt-tee: Real-time system availability for cyber-physical systems using arm trustzone. In *2022 IEEE Symposium on Security and Privacy (SP).* IEEE, 352–369.

[52] Zibo Wang, Yaofang Zhang, Yilu Chen, Hongri Liu, Bailing Wang, and Chonghua Wang. 2023. A survey on programmable logic controller vulnerabilities, attacks, detections, and forensics. *Processes* 11, 3 (2023), 918.

[53] Hyunguk Yoo and Irfan Ahmed. 2019. Control logic injection attacks on industrial control systems. In *ICT Systems Security and Privacy Protection: 34th IFIP TC 11 International Conference, SEC 2019, Lisbon, Portugal, June 25-27, 2019, Proceedings 34.* Springer, 33–48.

[54] Wei Zhou, Zhouqi Jiang, and Le Guan. 2023. Understanding MPU Usage in Microcontroller-based Systems in the Wild. In *Proceedings 2023 Workshop on Binary Analysis Research. San Diego, CA, USA: Internet Society.*

[55] Xia Zhou, Yujie Bu, Meng Xu, Yajin Zhou, and Lei Wu. 2024. uBOX: A Lightweight and Hardware-assisted Sandbox for Multicore Embedded Systems. *IEEE Transactions on Dependable and Secure Computing* (2024).

[56] Xia Zhou, Jiaqi Li, Wenlong Zhang, Yajin Zhou, Wenbo Shen, and Kui Ren. 2022. OPEC: operation-based security isolation for bare-metal embedded systems. In *Proceedings of the Seventeenth European Conference on Computer Systems.* 317–333.

[57] Ioannis Zografopoulos, Juan Ospina, Xiaorui Liu, and Charalambos Konstantinou. 2021. Cyber-physical energy systems security: Threat modeling, risk assessment, resources, metrics, and case studies. *IEEe Access* 9 (2021), 29775–29818.

A Appendices

A.1 Open Science

A.2 Ethical Considerations

A.3 A1

In the calculation of the anomaly detection threshold ($\lambda = k * \sigma$), the method based on the standard deviation (std) utilizes the complete dataset and performs exceptionally well when the data follows a normal distribution. Regarding the selection of the threshold multiplier k, a larger k value helps reduce the false positive rate but may increase the risk of FNR. Conversely, a smaller k enhances sensitivity to anomalies but can lead to a higher false positive rate. According to the $3 * \sigma$ rule, when k is set to 3, approximately 99.73% of the data falls within the range of the mean 3 standard deviations, theoretically resulting in high sensitivity to anomalies while maintaining a low false positive rate (around 0.27%). This rule is widely used in industrial quality control due to its well-established balance between detection sensitivity and system reliability.

Historical experimental results show that when $k = 2$, the false positive rate tends to be high, whereas $k = 4$ may significantly increase the likelihood of FNR. Considering both detection performance and system stability, $k = 3$ offers the best trade-off between FAR and FNR. Practical evaluations further support the rationality of this choice, as demonstrated in Table 9.

Control systems typically exhibit a certain degree of robustness, allowing them to automatically adjust and return to a stable state when subjected to short-term or transient deviations. However, if such deviations persist beyond a predefined duration threshold, the system's inherent recovery mechanisms may fail, making it difficult to reestablish stable operation. Therefore, selecting an appropriate sliding window size becomes crucial—not only to maintain system stability but also to reduce the overhead of unnecessary identity authentications triggered by FAR.

Table 9: Detection performance under different sigma thresholds

σ Size	Detection Rate	FAR	FNR
1	1.0000	0.2512	0.0000
2	1.0000	0.1302	0.0000
3	1.0000	0.0654	0.0000
4	0.9975	0.0289	0.0025

To this end, we evaluate the impact of different window sizes on the reduction of FAR, while fixing the detection threshold at 3σ to avoid false negatives rate (FNR). Experimental results show that as the window size increases, the false positive rate significantly decreases, while the detection rate remains consistently at 100%. Specifically, as the window size increases from 1 to 20, the false positive rate drops from 0.0654% to a minimum of 0.0006%, with the false negative rate consistently remaining at 0. This indicates that enlarging the window size effectively reduces the occurrence of FAR without compromising detection performance.

In the initial stage (window sizes from 1 to 5), increasing the window size imposes stricter criteria for identifying attacks—requiring multiple consecutive data points to exceed the threshold. Since random fluctuations in normal data rarely persist across multiple cycles, the false positive rate drops sharply. In the later stage (window sizes from 6 to 20), while the number of FAR remains relatively stable, the denominator in the false positive rate formula, $2000(240/w)$, decreases due to the reduced value of $240/w$ as w increases. Mathematically, as the window size increases, the numerator (number of FAR) tends to stabilize, but the shrinking denominator causes a slow rise in the false positive rate.

From a physical perspective, larger windows reduce the likelihood of misjudgment but also decrease the opportunities for identity authentication checks. This trade-off results in a non-monotonic variation in the FAR as the window size increases.

Table 10: Effect of Sliding Window Size on Detection and False Alarm Rates

Window Size	Detection Rate	False Alarm Rate
1	1.0000	0.0654
2	1.0000	0.0121
3	1.0000	0.0029
4	1.0000	0.0009
5	1.0000	0.0006
10	1.0000	0.0010
15	1.0000	0.0014
20	1.0000	0.0018